**ASSIGNMENT 1 [CO1]**
Course Outcome 1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs
**Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System**
Total Marks: 50 | Nature of Assignment: Individual | Duration: 1 Week
Bloom's Level Assessed: K5 – Evaluate

| Detail | Entry |
|---|---|
| Student name | A Shiva Shankar Reddy |
| Register number | 192425174 |
| Department | BTech-AIML |
| Course / subject | DSA0102/ C++ |
| Faculty name | Arun Kumar |
| Submission date | 20-10-2026 |
| GitHub repository | https://github.com/shivareddy1012/DSA0102-192425174-C.Plus.Plus |

**Contents**

Execution Notes and Assumptions 4
Execution environment used for this report 4
Assumptions (the assignment leaves these open) 4
Fare rules used throughout 4
Task 1 – Understand the Problem and Identify the Objects (4 Marks) 5
1.1 Requirement 5
1.2 Objects in the problem 5
1.3 Procedural style and object-oriented style 5
1.4 OOP principles and where they appear 6
1.5 Rules every correct journey must satisfy 6
1.6 Class diagram 7
1.7 Program for the class design 7
1.8 Alternative concept: purely procedural solution 10
1.9 Conclusion 11
Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks) 11
2.1 Requirement 11
2.2 The five kinds of tokens 11
2.3 #define, const, constexpr, enum and enum class 11
2.4 Constants program 12
2.5 Naming rules and conventions 14
2.6 The single set of constants 14
2.7 Alternative concept: #define macros 14
2.8 Conclusion 15
Task 3 – Choose Data Types and Check Type Compatibility (4 Marks) 15
3.1 Requirement 15
3.2 Data-type table 15
3.3 Program for sizes, limits, overflow and precision 16
3.4 Is int enough for a day's collection? 19
3.5 Type compatibility 19
3.6 Why money is stored in integer paise 20
3.7 Alternative concept: money as double rupees 20
3.8 Conclusion 20
Task 4 – Control Implicit Conversions (4 Marks) 21
4.1 Requirement 21
4.2 Conversion points in the metro program 21
4.3 Predict the output 21
4.4 Program (exactly as supplied) and actual output 21
4.5 Explanation of each output line 22
4.6 Narrowing and brace initialisation 23
4.7 Alternative concept: every conversion explicit 23
4.8 Conclusion 24
Task 5 – Write Correct Expressions with the Right Precedence (4 Marks) 24
5.1 Requirement 24
5.2 Precedence and associativity of the operators used 25
5.3 Expressions for the fare logic 25
5.4 x++ against ++x, and i = i++ 26
5.5 / and % with negative operands 27
5.6 Predict the output 27
5.7 Operator-by-operator explanation 27
5.8 Alternative concept: lookup table bool peak[24] 28

5.9 Conclusion 29
Task 6 – Design the Control Structures (4 Marks) 29
6.1 Requirement 29
6.2 Choice of control structures 29
6.3 Pseudocode 30
6.4 Complete the code: TODO 1 and TODO 2 31
6.5 Boundary test: exactly 10:00 AM and 8:00 PM 32
6.6 Alternative concept: array indexed by zone 34
6.7 Conclusion 35
Task 7 – From Procedural to Object-Oriented (4 Marks) 35
7.1 Requirement 35
7.2 Procedural version 35
7.3 Object-oriented version 36
7.4 How the class prevents a negative balance 38
7.5 Which version is easier to change when the minimum balance changes? 38
7.6 Alternative concept: public data member 39
7.7 Conclusion 40
Task 8 – Read and Print Data Safely (4 Marks) 40
8.1 Requirement 40
8.2 How the input handling works 40
8.3 Formatting money with iomanip 40
8.4 Program using cin 40
8.5 Alternative concept: getline() and stoi() with try/catch 42
8.6 Conclusion 44
Task 9 – Implement and Test the Fare Logic (4 Marks) 44
9.1 Requirement 44
9.2 Design of the program 44
9.3 Source code 45
9.4 Actual output 48
9.5 Test table with expected and actual results 49
9.6 Alternative concept: table of test cases in a loop 51
9.7 Conclusion 52
Task 10 – Compare Expectations with Execution (4 Marks) 52
10.1 Requirement 52
10.2 Assumptions and method 52
10.3 Source code 52
10.4 Actual output 54
10.5 Expected against observed 55
10.6 Discussion of the differences 55
10.7 Compiler warnings with -Wall -Wextra 56
10.8 Alternative concept: int, long long or double accumulator 56
10.9 Conclusion 57
Task 11 – Recommend and Defend the Final Design (4 Marks) 57
11.1 Requirement 57
11.2 Recommended design 57
11.3 How a fare revision is carried out 58
11.4 Limitations 58
11.5 Is the design correct for the whole network? 58
11.6 Alternative concept: fare table read from a configuration file 58
11.7 Final recommendation 61
Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge, 8 Marks) 61

12.1 Changed requirement 61
A. Impact Analysis (2 Marks)62
B. Evaluation of the Original Design (2 Marks) 62
C. Adaptation or Redesign (2 Marks) 64
D. Evidence-Based Defence (2 Marks) 70
12.2 Conclusion 71
Summary of Results and Final Conclusion 71

Execution Notes and Assumptions

Execution environment used for this report

Every program in this report was compiled and executed with **g++ 13.3.0 on Ubuntu 24.04.4 LTS (x86_64, one CPU core)**, using g++ -std=c++17 -Wall -Wextra. The output blocks, compiler messages, sizes and timings in the report are the real results of those runs. They were not typed by hand.

The programs were not run on an online compiler, and no screenshots could be captured in that environment. The yellow lines marked "Screenshot to be inserted" show where the screenshots of your own run belong. Run each program yourself, take the screenshot, and insert it. [Replace each yellow placeholder with a real screenshot]

Sizes (sizeof) and timings depend on the system. If your compiler or computer is different, the values you obtain may differ, and the report text that quotes them should be updated.

The predicted outputs in Tasks 4 and 5 were written before the programs were run, and the actual outputs were then compared with them.

The source code is in the folders Task01 to Task12 (see the README.md in the source package). The assignment requires a GitHub repository with at least one commit for each task. [Create the repository, commit task by task, and write the link on the cover page]

Assumptions (the assignment leaves these open)

| No. | Missing detail | Assumption used in this report |
|---|---|---|
| 1 | Concession percentages for students and senior citizens are not given | Student 50%, senior citizen 40%. Both are constants and can be changed in one place |
| 2 | Order of the surcharge and the concession | The surcharge is added first, then the concession is applied to the result. All fares are multiples of 10 paise, so every result is a whole number of paise |
| 3 | Average fare used for network totals (Task 3) | Rs 20 per journey (2000 paise) |
| 4 | Mix of journeys used for the totals in Task 10 | Six journey types in a cycle with three passenger categories (see Task 10, Section 10.2) |
| 5 | Weekend, pass and concession rules in Task 12 | Listed at the start of Task 12 |
| 6 | Pseudocode format | A common ALGORITHM / INPUT / OUTPUT / BEGIN–END format is used. The SIMATS standard document was not available to me |

Fare rules used throughout

Base fares: Inner Rs 10, Middle Rs 20, Outer Rs 30 (1000, 2000 and 3000 paise), charged by the farthest zone travelled.

Peak hours: 8:00 to before 10:00 and 17:00 to before 20:00, with a 20% surcharge. 8:00 and 17:00 are peak; 10:00 and 20:00 are off-peak.
At least Rs 50 (5000 paise) must remain on the card after the fare is deducted. All money is stored as integer paise.
Task 1 – Understand the Problem and Identify the Objects (4 Marks)
1.1 Requirement
Analyse the fare problem from the object-oriented point of view: the objects (card, passenger, journey, zone, fare table) with their data and behaviour, the procedural style against the object-oriented style, the four OOP principles and where each appears, and the rules every correct journey must satisfy. Each object and rule has to be justified. The alternative concept is a purely procedural solution.
1.2 Objects in the problem
The problem statement mentions a smart card, passengers, journeys, zones and fares. Each of these has its own data and its own rules, so each became a class or an enumeration.

| Object | Data | Behaviour | Why it is a separate object |
|---|---|---|---|
| SmartCard | card id, balance in paise (private) | canTravel(fare), deduct(fare), recharge(amount), balance() | The balance is the value the main rules protect (never below Rs 50 after a journey, never negative). Keeping the balance and the checks in one class stops other code from breaking those rules. |
| Passenger | name, category (regular, student, senior citizen) | concessionPercent() | The concession depends on the kind of passenger, so the passenger decides it. New categories can be added without touching the fare code. |
| Journey | entry zone, exit zone, hour, minute | farthestZone(), isPeak() | The fare depends on the farthest zone and on the time of travel, and both come from the journey and not from the card. |
| Zone | Inner, Middle, Outer | none (enumeration) | A zone has no behaviour, only a fixed set of names. An enum class is enough and is safer than a full class. |
| FareTable | base fares (1000, 2000, 3000 paise), peak surcharge 20%, minimum balance 5000 paise | fareFor(zone, peak, concession) | All fare values are kept in one place, which fixes the problem that fares were typed in several places in the first version. |

1.3 Procedural style and object-oriented style
In a procedural program the data and the functions are separate. The balance would be a global variable (or a plain variable passed everywhere) and functions such as fare(), canTravel() and deduct() would work on it. This is simple for a small program, but any function can change the balance directly. That is how a negative balance can appear on a card in the first version of the gate software: one forgotten check is enough.
In the object-oriented style the balance and the functions that use it are put together in the class SmartCard. The balance is private, so the only way to change it is through deduct() or recharge(), and these functions apply the rules every time. The fare values live in FareTable,

the concession in Passenger, and the time rules in Journey. When a rule changes, there is one obvious class to change.

1.4 OOP principles and where they appear

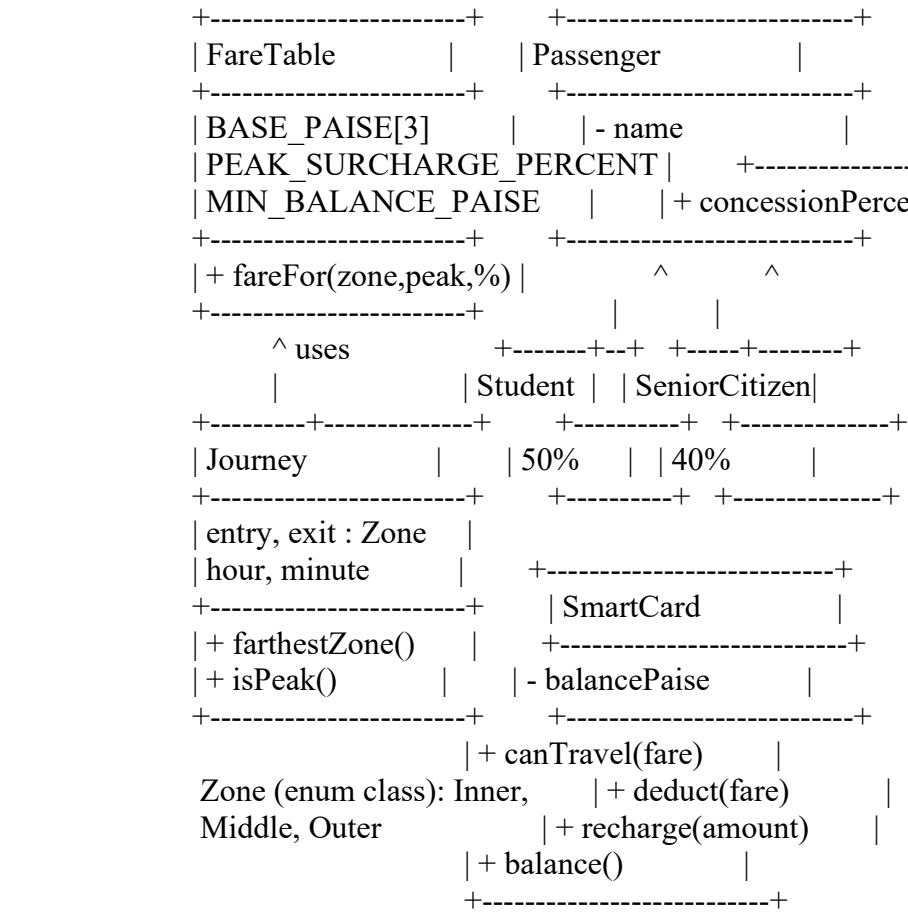
| Principle | Where it appears in this system | Evidence |
|---|---|---|
| Encapsulation | The balance of SmartCard is private. Only canTravel(), deduct() and recharge() can change it. | Task 7 (the private member cannot be changed from main) |
| Abstraction | The gate code calls fareFor() and deduct() without knowing how the surcharge or the minimum balance is calculated. | Task01_ClassDesign.cpp, main() |
| Inheritance | Student and SeniorCitizen are derived from Passenger and reuse its name and interface. | Task01_ClassDesign.cpp |
| Polymorphism | concessionPercent() is virtual. The same call through a Passenger pointer gives 0%, 50% or 40% depending on the real object. | Program output below: same journey, three different fares |

1.5 Rules every correct journey must satisfy

| No. | Rule | Why it is needed |
|---|---|---|
| R1 | After the fare is deducted at least Rs 50 (5000 paise) must remain on the card. | Given in the problem. It is the main business rule. |
| R2 | The balance must never be negative. | A negative balance appeared in the first version. R1 already implies it, but it is checked on its own for recharge and refunds too. |
| R3 | The fare depends on the farthest zone travelled. | Given in the problem. A journey from Inner to Outer is charged as Outer. |
| R4 | Peak hours are 8:00 to before 10:00 and 17:00 to before 20:00. 8:00 and 17:00 are peak; 10:00 and 20:00 are not. | The first version treated 10 AM and 8 PM inconsistently. One half-open interval rule removes the doubt. |
| R5 | The 20% surcharge is added with integer arithmetic on paise. | Avoids fares that are off by a paisa. |
| R6 | Students and senior citizens get a concession. The assignment does not give the percentages, so this report assumes 50% for students and 40% for senior citizens. | The percentages are kept in constants so they can be changed in one place. |
| R7 | The zone must be a known zone; an unknown zone gives no fare and the journey is refused. | Prevents a free journey caused by a wrong zone code. |
| R8 | A recharge must be a positive amount. | A negative recharge would reduce the balance without the minimum balance check. |
| R9 | A fare is deducted only after canTravel() has returned true. | Keeps the check and the deduction together. |

1.6 Class diagram

A simple text-based class diagram of the design (- means private, + means public):




1.7 Program for the class design

The following program is a small, working skeleton of the design. It was compiled and executed; it shows polymorphism (the same journey costs a different fare for each passenger type) and the SmartCard rule.

**C++ source code (file: Task01/Task01_ClassDesign.cpp)**

```
// Task 1: skeleton of the object-oriented design of the metro fare system
#include <iomanip>
#include <iostream>
#include <string>
#include <utility>

enum class Zone { Inner, Middle, Outer };

// FareTable: the only place where fares and limits are stored
class FareTable {
public:
    static constexpr long long BASE_PAISE[3] = {1000, 2000, 3000};
    static constexpr int PEAK_SURCHARGE_PERCENT = 20;
    static constexpr long long MIN_BALANCE_PAISE = 5000;

    static long long fareFor(Zone z, bool peak, int concessionPercent) {
        long long f = BASE_PAISE[static_cast<int>(z)];
        if (peak) f += f * PEAK_SURCHARGE_PERCENT / 100;
```

```
        return f * (100 - concessionPercent) / 100;
    }
};

// Passenger: base class, derived classes give different concessions (polymorphism)
class Passenger {
public:
    explicit Passenger(std::string n) : name(std::move(n)) {}
    virtual ~Passenger() = default;
    virtual int concessionPercent() const { return 0; }
    const std::string& getName() const { return name; }
private:
    std::string name;
};

class Student : public Passenger {
public:
    using Passenger::Passenger;
    int concessionPercent() const override { return 50; }  // assumed value
};

class SeniorCitizen : public Passenger {
public:
    using Passenger::Passenger;
    int concessionPercent() const override { return 40; }  // assumed value
};

// Journey: one trip, with the rule for the farthest zone and the peak check
struct Journey {
    Zone entry;
    Zone exit;
    int hour;
    int minute;

    Zone farthestZone() const {
        return static_cast<int>(entry) > static_cast<int>(exit) ? entry : exit;
    }
    bool isPeak() const {
        int m = hour * 60 + minute;
        return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
    }
};

// SmartCard: balance is private, so every change goes through the rules
class SmartCard {
public:
    explicit SmartCard(long long initialPaise) : balancePaise(initialPaise) {}
    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise - fare >= FareTable::MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
```

```
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
    long long balance() const { return balancePaise; }
private:
    long long balancePaise;
};

void printRs(long long paise) {
    std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}

int main() {
    SmartCard card(10000);  // Rs 100.00
    Passenger regular("Asha");
    Student student("Ravi");
    SeniorCitizen senior("Meena");
    const Passenger* people[] = {&regular, &student, &senior};
    Journey j{Zone::Inner, Zone::Outer, 9, 0};  // 9:00 AM, peak

    for (const Passenger* p : people) {
        long long f = FareTable::fareFor(j.farthestZone(), j.isPeak(), p->concessionPercent());
        std::cout << p->getName() << " pays ";
        printRs(f);
        std::cout << '\n';
    }
    long long f = FareTable::fareFor(j.farthestZone(), j.isPeak(), regular.concessionPercent());
    std::cout << "Card can travel: " << std::boolalpha << card.canTravel(f) << '\n';
    card.deduct(f);
    std::cout << "Balance after journey: ";
    printRs(card.balance());
    std::cout << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task01_ClassDesign Task01_ClassDesign.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**

```
Asha pays Rs 36.00
Ravi pays Rs 18.00
Meena pays Rs 21.60
Card can travel: true
Balance after journey: Rs 64.00
```

[Screenshot to be inserted: Task01_Output.png - code and output from the compiler window]
**Check by hand:** the journey is Inner to Outer at 9:00 AM, so the farthest zone is Outer and it is peak. Fare = 3000 + 3000 × 20 / 100 = 3600 paise = Rs 36.00 for a regular passenger. The student pays 3600 × 50 / 100 = 1800 paise (Rs 18.00). The senior citizen pays 3600 × 60 / 100 = 2160 paise (Rs 21.60). The card has 10000 paise, and 10000 − 3600 = 6400 ≥ 5000, so the

journey is allowed and the balance becomes Rs 64.00. All values agree with the program output.

1.8 Alternative concept: purely procedural solution

If the system were written with global variables and free functions only, it would look like this (the full sketch is in Task01_Procedural_sketch.cpp):

**C++ source code (file: Task01/Task01_Procedural_sketch.cpp)**

```
// Task 1 (alternative concept): the same idea in a purely procedural style
#include <iostream>

long long balancePaise = 10000;  // global state, any function can change it
const long long BASE_PAISE[3] = {1000, 2000, 3000};

long long fare(int zone, bool peak, int concessionPercent) {
   long long f = BASE_PAISE[zone];
   if (peak) f += f * 20 / 100;
   return f * (100 - concessionPercent) / 100;
}

bool canTravel(long long f) { return balancePaise - f >= 5000; }

int main() {
   long long f = fare(2, true, 0);  // Outer zone, peak, no concession
   if (canTravel(f)) balancePaise -= f;
   std::cout << "Balance: " << balancePaise << " paise\n";
}
```

Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
Balance: 6400 paise
```

| Point | Object-oriented (classes) | Procedural (globals and functions) |
|---|---|---|
| Time per journey | O(1): a few comparisons and arithmetic operations | O(1): the same operations |
| Memory per card | O(1): one long long balance (8 bytes on this system) plus the card id | O(1): one global long long. With many cards it would be an array of balances, still O(1) per card |
| Who can change the balance | Only SmartCard member functions | Any function in the program |
| Change in one fare rule (for example the surcharge changes from 20% to 25%) | One constant in FareTable | One constant if the programmer was careful, but nothing forces it. In the hurried first version the values were typed in several places, and every place had to be found and edited |
| Change in the rule of a new passenger type | Add one derived class | Add a new branch in every function that handles concessions |

The time per journey and the memory per card do not change between the two styles (both are O(1)). I did not measure them separately because the work done per journey is the same. The real difference is in maintenance: the amount of code that must be edited and checked when a rule changes is smaller and easier to find in the object-oriented design.

1.9 Conclusion

Five objects were identified and each one was justified by the data it owns and the rules it protects. The object-oriented design is chosen because it keeps the balance rules in one class and the fare values in one table, without any extra cost in time or memory.

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

2.1 Requirement

Define the fares, surcharge and limits so that a revision is made in one place only. Discuss the five kinds of tokens, #define, const and constexpr, enum and enum class, and the naming rules. The alternative concept is writing the fares with #define macros.

2.2 The five kinds of tokens

The compiler splits the source code into tokens. The examples below are taken from the fare program of Task 6.

| **Token kind** | **Meaning** | **Examples from the program** |
|---|---|---|
| Keywords | Words reserved by the language | enum, class, constexpr, switch, case, default, break, return, if, else, for, bool, int, long, const, true, false |
| Identifiers | Names given by the programmer | Zone, Inner, BASE_PAISE, PEAK_SURCHARGE_PERCENT, MIN_BALANCE_PAISE, fare, canTravel, balance, trips |
| Constants (literals) | Fixed values written in the code | 1000, 2000, 20, 5000, 10000, 100, false, true, '0', '\n', "Denied (low balance)" |
| Operators | Symbols that perform an operation | =, +=, -=, *, /, %, >=, &&, ::, <<, ++ |
| Punctuators | Symbols that separate or group | ; , ( ) { } [ ] : and # |

2.3 #define, const, constexpr, enum and enum class

| **Feature** | **Type checked?** | **Scope** | **Compile-time value?** | **Remarks for the fare table** |
|---|---|---|---|---|
| #define | No. It is text replacement by the preprocessor | None. It ignores namespaces and blocks | Yes, but only as text | Dangerous with expressions. Names do not appear in the compiler or debugger |
| const | Yes | Normal C++ scope | Only if initialised with a constant expression | Safe, but the compiler does not guarantee a compile-time value |
| constexpr | Yes | Normal C++ scope | Guaranteed. The compiler gives an error otherwise | Best choice for fares, percentages and limits |

| Feature | Type checked? | Scope | Compile-time value? | Remarks for the fare table |
|---|---|---|---|---|
| enum | Weak. Converts to int silently | Names leak into the enclosing scope | Yes | Allows comparison with plain numbers and with other enums |
| enum class | Strong. No implicit conversion | Names stay inside the enum (Zone::Inner) | Yes | Best choice for zones |

**Why constexpr is preferred for the fare table.** A constexpr value is checked by the compiler, has a real type (long long for paise), obeys scope rules and is guaranteed to be known at compile time, so it can be used for array sizes and in static_assert. A const variable may only get its value at run time (for example const int n = std::stoi(text);), and a macro has no type at all.
**Why enum class is safer.** With a plain enum the value OLD_OUTER can be compared with the number 2 or with a value from another enum without any warning. With enum class the program has to say static_cast<int>(z) when it really wants the number. The program below shows both, and the next program shows what the compiler says when the conversion is not written.
2.4 Constants program
**C++ source code (file: Task02/Task02_Constants.cpp)**

```
// Task 2: tokens, constants and enums
#include <iostream>

#define FARE_INNER_MACRO 1000          // macro: no type, no scope
#define DOUBLE_IT(x) x * 2             // careless macro

const int SURCHARGE_CONST = 20;          // const: typed, but not always a compile-time constant
constexpr int PEAK_SURCHARGE_PERCENT = 20;        // constexpr: guaranteed compile-time constant
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int ZONE_COUNT = sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]);

enum class Zone { Inner, Middle, Outer };   // scoped, no implicit conversion to int
enum OldZone { OLD_INNER, OLD_MIDDLE, OLD_OUTER };  // plain enum: names leak, converts to int

namespace Gate {
constexpr int LIMIT = 5;
}

int main() {
    // a constexpr value can be used where a compile-time constant is needed
    int zoneNames[ZONE_COUNT] = {0, 0, 0};
    std::cout << "ZONE_COUNT = " << ZONE_COUNT << ", array size = "
              << sizeof(zoneNames) / sizeof(zoneNames[0]) << '\n';

    // macro pitfall
    std::cout << "DOUBLE_IT(3 + 1) = " << DOUBLE_IT(3 + 1) << " (expected 8)\n";
```

```
    // plain enum converts silently, enum class does not
    std::cout << "OLD_OUTER == 2 : " << (OLD_OUTER == 2) << '\n';
    std::cout << "OLD_INNER < 5  : " << (OLD_INNER < 5) << '\n';
    Zone z = Zone::Outer;
    std::cout << "Outer index (needs cast) = " << static_cast<int>(z) << '\n';
    std::cout << "Outer fare = " << BASE_PAISE[static_cast<int>(z)] << " paise\n";

    // macro has no scope and no type
    std::cout << "Macro fare / 3 = " << FARE_INNER_MACRO / 3 << '\n';
    std::cout << "Gate::LIMIT = " << Gate::LIMIT << ", SURCHARGE_CONST = " <<
SURCHARGE_CONST << '\n';
    std::cout << "Surcharge on Rs 30 = " << BASE_PAISE[2] *
PEAK_SURCHARGE_PERCENT / 100
              << " paise, minimum balance = " << MIN_BALANCE_PAISE << " paise\n";
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task02_Constants Task02_Constants.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
ZONE_COUNT = 3, array size = 3
DOUBLE_IT(3 + 1) = 5 (expected 8)
OLD_OUTER == 2 : 1
OLD_INNER < 5  : 1
Outer index (needs cast) = 2
Outer fare = 3000 paise
Macro fare / 3 = 333
Gate::LIMIT = 5, SURCHARGE_CONST = 20
Surcharge on Rs 30 = 600 paise, minimum balance = 5000 paise
```

[Screenshot to be inserted: Task02_Output.png - code and output from the compiler window]

**Analysis of the output.** DOUBLE_IT(3 + 1) printed 5 and not 8, because the macro only replaces text: 3 + 1 * 2 is 3 + 2. OLD_OUTER == 2 and OLD_INNER < 5 both give 1 (true), so the plain enum behaves like a number. The enum class value needed static_cast<int> to be used as an array index. ZONE_COUNT (a constexpr) was usable as an array size.

Code that enum class rejects

**C++ source code (file: Task02/Task02_COMPILEFAIL.cpp)**

```
// Task 2: code that enum class rejects (expected to fail to compile)
enum class Zone { Inner, Middle, Outer };

int main() {
    Zone z = Zone::Inner;
    int n = z;          // error: no implicit conversion
    if (z == 0) {}      // error: cannot compare with int
    return n;
}
```

**Compiler messages with -Wall -Wextra (compilation failed, as intended)**

```
Task02_COMPILEFAIL.cpp: In function 'int main()':
Task02_COMPILEFAIL.cpp:6:13: error: cannot convert 'Zone' to 'int' in initialization
```

```
    6 |     int n = z;          // error: no implicit conversion
      |             ^
      |             |
      |            Zone
Task02_COMPILEFAIL.cpp:7:11: error: no match for 'operator==' (operand types are 'Zone' and 'int')
    7 |     if (z == 0) {}      // error: cannot compare with int
      |        ~ ^~ ~
      |        |   |
      |        Zone int
```

Both lines were rejected at compile time, so these mistakes cannot reach the running gate software.

2.5 Naming rules and conventions

An identifier may contain letters, digits and underscore, and must not start with a digit.

It must not be a keyword, and C++ is case sensitive (fare and Fare are different names).

Names starting with an underscore followed by a capital letter, or containing two underscores in a row, are reserved for the compiler and library, so they are avoided.

Conventions used in this report: constants in UPPER_SNAKE_CASE (BASE_PAISE), types and classes in PascalCase (SmartCard, Zone), functions and variables in camelCase (canTravel, balancePaise). Money names carry the unit (...Paise) so that rupees and paise are not mixed by mistake.

2.6 The single set of constants

All fares and limits are placed together at the top of the program. A fare revision means changing one number in this block:

```
enum class Zone { Inner, Middle, Outer };              // add Airport here for a fourth zone

constexpr int  BASE_PAISE[] = {1000, 2000, 3000};      // Rs 10, Rs 20, Rs 30
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  STUDENT_CONCESSION_PERCENT = 50;        // assumed
constexpr int  SENIOR_CONCESSION_PERCENT = 40;         // assumed
constexpr long long MIN_BALANCE_PAISE = 5000;          // Rs 50 must remain
```

For a fourth zone, a new enumerator is added to Zone and a new value is added to BASE_PAISE. This is shown in Task 12 where the table is indexed by the zone and its size comes from Zone::Count.

2.7 Alternative concept: #define macros

If the fares were written as macros, the code would look like this:

```
#define FARE_INNER   1000
#define FARE_MIDDLE  2000
#define FARE_OUTER   3000
#define PEAK_SURCHARGE_PERCENT 20
#define ZONE_INNER   0      // zones as plain numbers instead of enum class
#define ZONE_MIDDLE  1
#define ZONE_OUTER   2
```

| Point | constexpr + enum class (used) | #define macros (alternative) |
|---|---|---|
| Type checking | Fares are long long, zones are of type Zone. Wrong conversions are errors (shown above) | No type. The macro 1000 is just text, and a zone is any int, so a fare can be passed as a zone without warning |
| Scope | Normal C++ scope, can be inside a namespace or class | None. The name is global from its line to the end of the file and can clash with any other name |
| Errors and debugging | Compiler messages and debugger show the name | The compiler only sees the replaced text. Messages mention numbers, not names |
| Memory | A scalar constexpr usually needs no storage because the value is used directly. The array BASE_PAISE needs a small read-only block when it is indexed with a variable | A macro occupies no storage; the number is placed directly in the instructions |
| Running time | No effect. Values are known at compile time. Using one is O(1) | No effect. O(1) |
| Maintenance | One place to change, checked by the compiler | One place to change, but unchecked and easy to misuse (see DOUBLE_IT) |

2.8 Conclusion

constexpr for numbers and enum class for zones are chosen. They cost nothing at run time, they give type checking and scope, and the compiler rejected the unsafe conversions in the test above. The macro version has the same speed and memory but none of the safety.

Task 3 – Choose Data Types and Check Type Compatibility (4 Marks)

3.1 Requirement

Select a data type for every value in the system, show sizeof on the system used, explain why money is stored in integer paise, explain type compatibility, and use <climits> to decide whether int is enough for a day's collection across the network. The alternative concept is storing money as double rupees.

3.2 Data-type table

The sizes below are the values printed by the program in Section 3.3 on the g++ 13.3.0 / x86_64 Linux system used for this report. They can differ on other systems.

| Value | Type chosen | sizeof | Range (this system) | Justification |
|---|---|---|---|---|
| Fare (paise) | long long | 8 | −9.22×10^18 to 9.22×10^18 | A single fare would fit in int, but the same type as balances and totals avoids mixed arithmetic |
| Card balance (paise) | long long | 8 | same | Money in integer paise: exact. No fractional part |
| Day total / network total (paise) | long long | 8 | same | Over 2.1 billion paise at network level (see 3.4) |

| Value | Type chosen | sizeof | Range (this system) | Justification |
|---|---|---|---|---|
| Hour of day | int | 4 | −2,147,483,648 to 2,147,483,647 (0 to 23 used) | Natural type for integers. A smaller type would save no useful space |
| Minute | int | 4 | same (0 to 59 used) | Same reason as hour |
| Zone | enum class Zone | 4 | only Inner, Middle, Outer | Type safe. The underlying type is int here |
| Zone number typed by the operator | int | 4 | same as int | Checked to be 0 to 2 before use |
| Journey count per day | int | 4 | up to 2,147,483,647 | 3,000,000 fits. A yearly count would need long long |
| Peak flag | bool | 1 | true or false | A yes/no answer |
| Zone letter (I, M, O) | char | 1 | −128 to 127 (signed char here) | One character for display |
| Surcharge / concession percent | int | 4 | same as int | Whole percentages only |
| Card id | unsigned int | 4 | 0 to 4,294,967,295 | An id is never negative |
| Rupees for display only | double | 8 | about ±1.8×10^308, 15–16 digits | Only used to print, never to store or add money |
| float | not used | 4 | about 7 digits | Too few digits for money |

auto is not used for money values, because auto x = 5 / 2.0; silently becomes double, and a type written explicitly is easier to review. auto is acceptable for loop variables such as for (const auto& t : trips) where the type is obvious from the array.

3.3 Program for sizes, limits, overflow and precision

**C++ source code (file: Task03/Task03_DataTypes.cpp)**

```
// Task 3: data types, sizes, limits, overflow and precision
#include <climits>
#include <cmath>
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

int main() {
    // one variable for each kind of value used in the system
    long long farePaise = 3600;
    long long balancePaise = 10000;
    long long dayTotalPaise = 0;
    int hour = 17;
    int minute = 30;
    Zone zone = Zone::Outer;
```

```
    int zoneNumber = 2;
    int journeysPerDay = 20000;
    bool peak = true;
    char zoneLetter = 'O';
    int surchargePercent = 20;
    double rupeesForDisplay = 36.0;
    float f = 36.0f;
    unsigned int cardId = 1000001u;

    std::cout << "--- sizeof on this system ---\n";
    std::cout << "farePaise (long long)    : " << sizeof(farePaise) << '\n';
    std::cout << "balancePaise (long long) : " << sizeof(balancePaise) << '\n';
    std::cout << "dayTotalPaise (long long): " << sizeof(dayTotalPaise) << '\n';
    std::cout << "hour (int)               : " << sizeof(hour) << '\n';
    std::cout << "minute (int)             : " << sizeof(minute) << '\n';
    std::cout << "zone (enum class Zone)   : " << sizeof(zone) << '\n';
    std::cout << "zoneNumber (int)         : " << sizeof(zoneNumber) << '\n';
    std::cout << "journeysPerDay (int)     : " << sizeof(journeysPerDay) << '\n';
    std::cout << "peak (bool)              : " << sizeof(peak) << '\n';
    std::cout << "zoneLetter (char)        : " << sizeof(zoneLetter) << '\n';
    std::cout << "surchargePercent (int)   : " << sizeof(surchargePercent) << '\n';
    std::cout << "rupeesForDisplay (double): " << sizeof(rupeesForDisplay) << '\n';
    std::cout << "f (float)                : " << sizeof(f) << '\n';
    std::cout << "cardId (unsigned int)    : " << sizeof(cardId) << '\n';

    std::cout << "\n--- limits from <climits> ---\n";
    std::cout << "INT_MIN   = " << INT_MIN << '\n';
    std::cout << "INT_MAX   = " << INT_MAX << '\n';
    std::cout << "UINT_MAX  = " << UINT_MAX << '\n';
    std::cout << "LLONG_MIN = " << LLONG_MIN << '\n';
    std::cout << "LLONG_MAX = " << LLONG_MAX << '\n';

    std::cout << "\n--- network total (assumption: every journey costs Rs 20 = 2000 paise) ---
\n";
    const long long journeys[] = {20000, 500000, 3000000};
    for (long long n : journeys) {
        long long total = n * 2000;
        std::cout << std::setw(8) << n << " journeys: " << std::setw(12) << total << " paise"
                  << "  fits in int: " << std::boolalpha << (total <= INT_MAX);
        if (total > INT_MAX)
            std::cout << "  (int would hold " << static_cast<int>(total) << ')';
        std::cout << '\n';
    }

    std::cout << "\n--- type compatibility ---\n";
    std::cout << "5 / 2          = " << 5 / 2 << '\n';
    std::cout << "5 / 2.0        = " << 5 / 2.0 << '\n';
    std::cout << "true + true    = " << true + true << '\n';
    std::cout << "'A' + 1        = " << 'A' + 1 << '\n';
    std::cout << "sizeof(5 + 2.0) = " << sizeof(5 + 2.0) << '\n';
```

```
    std::cout << "\n--- precision problems with double ---\n";
    std::cout << std::setprecision(17);
    std::cout << "0.1 + 0.2 = " << 0.1 + 0.2 << '\n';
    std::cout << "0.1 + 0.2 == 0.3 : " << std::boolalpha << (0.1 + 0.2 == 0.3) << '\n';
    std::cout << "0.29 * 100 = " << 0.29 * 100 << '\n';
    std::cout << "static_cast<int>(0.29 * 100) = " << static_cast<int>(0.29 * 100)
              << "  (a paisa lost)\n";

    std::cout << "\n--- error when Rs 0.10 is added n times (double rupees vs integer paise) ---
\n";
    const long long counts[] = {1000, 100000, 1000000, 10000000};
    std::cout << std::setprecision(6) << std::scientific;
    for (long long n : counts) {
        double sum = 0.0;
        long long paise = 0;
        for (long long i = 0; i < n; ++i) {
            sum += 0.10;
            paise += 10;
        }
        double exact = static_cast<double>(paise) / 100.0;
        std::cout << "n = " << std::setw(9) << std::defaultfloat << n << std::scientific
                  << "  |double sum - exact| = " << std::fabs(sum - exact)
                  << "   paise total = " << std::defaultfloat << paise << '\n' << std::scientific;
    }
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task03_DataTypes Task03_DataTypes.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
--- sizeof on this system ---
farePaise (long long)    : 8
balancePaise (long long) : 8
dayTotalPaise (long long): 8
hour (int)               : 4
minute (int)             : 4
zone (enum class Zone)   : 4
zoneNumber (int)         : 4
journeysPerDay (int)     : 4
peak (bool)              : 1
zoneLetter (char)        : 1
surchargePercent (int)   : 4
rupeesForDisplay (double): 8
f (float)                : 4
cardId (unsigned int)    : 4

--- limits from <climits> ---
INT_MIN   = -2147483648
INT_MAX   = 2147483647
UINT_MAX  = 4294967295
LLONG_MIN = -9223372036854775808
```

```
LLONG_MAX = 9223372036854775807

--- network total (assumption: every journey costs Rs 20 = 2000 paise) ---
   20000 journeys:     40000000 paise  fits in int: true
  500000 journeys:   1000000000 paise  fits in int: true
 3000000 journeys:   6000000000 paise  fits in int: false  (int would hold 1705032704)

--- type compatibility ---
5 / 2          = 2
5 / 2.0        = 2.5
true + true    = 2
'A' + 1        = 66
sizeof(5 + 2.0) = 8

--- precision problems with double ---
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 : false
0.29 * 100 = 28.999999999999996
static_cast<int>(0.29 * 100) = 28  (a paisa lost)

--- error when Rs 0.10 is added n times (double rupees vs integer paise) ---
n =      1000  |double sum - exact| = 1.406875e-12   paise total = 10000
n =    100000  |double sum - exact| = 1.884837e-08   paise total = 1000000
n =   1000000  |double sum - exact| = 1.332883e-06   paise total = 10000000
n =  10000000  |double sum - exact| = 1.610246e-04   paise total = 100000000
```

[Screenshot to be inserted: Task03_Output.png - code and output from the compiler window]

3.4 Is int enough for a day's collection?

**Assumption:** every journey costs Rs 20 (2000 paise). This is a simple average chosen only to estimate the size of the totals; it is not a figure from the assignment. (Task 10 repeats the calculation with a mixed set of fares.)

| **Journeys per day** | **Total (paise)** | **Total (Rs)** | **Fits in int (max 2,147,483,647)?** |
|---|---|---|---|
| 20,000 (one station) | 40,000,000 | 4,00,000 | Yes |
| 500,000 (one line) | 1,000,000,000 | 1,00,00,000 | Yes, but already about 47% of the int range |
| 3,000,000 (whole network) | 6,000,000,000 | 6,00,00,000 | No. Overflow. The program shows that an int would hold 1,705,032,704, a wrong value that looks normal |

An int is enough for one station and for one line under this assumption, but not for the whole network. For the line, a small increase in the average fare would already be a problem: 2,147,483,647 ÷ 500,000 is about 4,294 paise, so an average above about Rs 42.94 per journey would overflow. long long (maximum about 9.22×10^18) is therefore used everywhere for money. Note that signed overflow is undefined behaviour in C++; the value shown for int comes from converting the correct long long total with static_cast<int>, which on g++ wraps around.

3.5 Type compatibility

When values of different types appear in one expression, the compiler applies the usual arithmetic conversions: bool, char and short are first promoted to int; if one operand is double, the other is converted to double; if one is long long and the other int, the result is long long; if a signed and an unsigned operand of the same size meet, the signed value is converted to unsigned (the surprise examined in Task 4). The program output confirms: 5 / 2 is 2 (both int), 5 / 2.0 is 2.5, true + true is 2 (bool promoted to int), 'A' + 1 is 66 (char promoted to int) and sizeof(5 + 2.0) is 8, which shows that the int became a double.

3.6 Why money is stored in integer paise

Most decimal fractions such as 0.1 cannot be stored exactly in binary floating point. The output above shows 0.1 + 0.2 printed as 0.30000000000000004, 0.1 + 0.2 == 0.3 is false, and 0.29 * 100 is 28.999999999999996, so static_cast<int>(0.29 * 100) gives 28. A fare converted to paise this way would lose one paisa, which matches the complaint that some fares were off by a paisa. Integer paise are exact, comparisons with == are reliable, and the arithmetic is fast.

3.7 Alternative concept: money as double rupees

If money were kept as double rupees, a fare of Rs 36 would be 36.0 and the surcharge would be fare * 1.2. The balance check would be balance - fare >= 50.0, which is risky because of rounding (a balance that should leave exactly Rs 50.00 might leave 49.99999999999999). The comparison would need a tolerance such as >= 50.0 - 1e-9.

The last part of the program above measures the error when Rs 0.10 is added n times, once in double rupees and once in integer paise (the paise result is exact). The numbers in the table are taken from the output.

| n (additions of Rs 0.10) | Absolute error of the double sum (Rs) | Error in integer paise |
|---|---|---|
| 1,000 | 1.41×10^-12 | 0 |
| 100,000 | 1.88×10^-8 | 0 |
| 1,000,000 | 1.33×10^-6 | 0 |
| 10,000,000 | 1.61×10^-4 | 0 |

The error grows faster than n. From 1,000 to 100,000 additions (100 times more) it grew about 13,000 times, and from 100,000 to 10,000,000 (100 times more) about 8,500 times, so in these runs it grew roughly like n squared. This is plausible because the running sum becomes larger, so each addition rounds at a coarser level. This is a description of what was measured on this system, not a general proof.

| Point | long long paise | double rupees |
|---|---|---|
| Time per addition | O(1) | O(1) (measured in Task 10: roughly 1 ns for long long and 3 ns for double at -O0 on this system; the exact values vary between runs) |
| Space per value | 8 bytes | 8 bytes |
| Exactness | Exact up to about 9.2×10^18 paise | Rounding error that grows with the number of additions |
| Comparisons such as balance >= 50 | Exact | Need a tolerance |

3.8 Conclusion

Integer paise stored in long long is the correct choice. The evidence is: int overflows for the whole network (6,000,000,000 paise against a limit of 2,147,483,647) and double rupees gave a lost paisa (0.29 * 100 becomes 28) and an error that grows with n. Both problems are resolved by long long paise.

Task 4 – Control Implicit Conversions (4 Marks)

4.1 Requirement

Identify the places where C++ converts a value without being told: integer division and integer-to-double conversion, signed/unsigned comparison and its warning, narrowing conversions and brace initialisation, and static_cast. Predict the output of the supplied program before running it, then compare. The alternative concept is writing every conversion explicitly.

4.2 Conversion points in the metro program

| **Place in the program** | **Conversion** | **Risk or surprise** |
|---|---|---|
| f * PEAK_SURCHARGE_PERCENT / 100 (long long × int) | int is converted to long long | None. The wider type wins. Multiplying before dividing keeps the result exact |
| f * (100 - pct) / 100 | Integer division | Writing f * ((100 - pct) / 100) would give 0 for any pct above 0 because the division happens first |
| fare / 100.0 when printing | long long to double | Fine for printing only. Never store the result as money |
| hour * 60 + minute | int arithmetic | None, values are small |
| bool peak = (i % 6) >= 3; | Comparison gives bool | None |
| BASE_PAISE[static_cast<int>(z)] | Explicit cast of enum class to int | The cast is required, so it is visible to the reader |
| Comparing an int balance with an unsigned value | int converted to unsigned | A negative balance becomes a huge positive number. This is the sign-compare surprise |
| int x = 3.7; or int y = bigLongLong; | Narrowing | Fraction or high bits are lost silently |

4.3 Predict the output

**My predicted output (written before running the program):**

3 3.5
C 68
1 true
s >= u

Reasoning used for the prediction: a / b is 7 / 2 with both operands int, so the result is 3, and 3 is then converted to double 3.0, which cout prints as 3. The second value is 7.0 / 2 = 3.5. 'A' + 2 is 67, stored in a char as 'C'. ch + 1 promotes ch to int, so it prints 68. A bool initialised with 5 becomes true, which prints as 1, and after boolalpha as true. In s < u the int s (−1) is converted to unsigned int, giving 4294967295, which is not less than 1, so the result is "s >= u".

4.4 Program (exactly as supplied) and actual output

**C++ source code (file: Task04/Task04.cpp)**

#include <iostream>

```
int main()
{
   int a = 7, b = 2;
   double c = a / b; double d = static_cast<double>(a) / b;
   std::cout << c << ' ' << d << '\n';
   char ch = 'A' + 2;
   std::cout << ch << ' ' << ch + 1 << '\n';
   bool flag = 5;
   std::cout << flag << ' ' << std::boolalpha << flag << '\n';
   unsigned int u = 1;
   int s = -1;
   std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task04 Task04.cpp

**Compiler messages with -Wall -Wextra**

```
Task04.cpp: In function 'int main()':
Task04.cpp:13:21: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
   13 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
      |                   ~~^~~
```

**Actual output (from execution)**

```
3 3.5
C 68
1 true
s >= u
```

[Screenshot to be inserted: Task04_Output.png - code and output from the compiler window]

**Comparison with the prediction:** there is no difference between the predicted and the actual output. The compiler did give a warning that was not part of the output: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]. It points to the last cout line and warns about exactly the conversion that produces the surprising "s >= u".

4.5 Explanation of each output line

| Output line | What happened | Conversion or rule |
|---|---|---|
| 3 3.5 | c holds 3 and d holds 3.5 | a / b is integer division (7 / 2 = 3, the .5 is dropped) and only then converted to double 3.0. For d, static_cast<double>(a) is done first, so the division is done in double |
| C 68 | 'A' + 2 = 67 is stored as 'C'. ch + 1 printed as a number | Integral promotion: char is promoted to int in arithmetic. The << operator then prints an int. ch alone is printed as a character |
| 1 true | flag holds true. First printed as 1, then as true | int to bool conversion: any non-zero value becomes true. boolalpha changes how bools are printed, and it stays in effect for later output |
| s >= u | The comparison says −1 is not less than 1 | Signed to unsigned conversion: −1 becomes 4294967295 (UINT_MAX). The compiler warns with -Wsign-compare |

4.6 Narrowing and brace initialisation

Narrowing means converting to a type that cannot hold every value of the source type. With = it is allowed silently. With braces it is a compile-time error. Both cases were tried:

**C++ source code (file: Task04/Task04_narrowing.cpp)**

```
// Task 4: the same narrowing with = compiles, and loses the fraction
#include <iostream>
int main()
{
   int x = 3.7;
   long long big = 3000000000LL;
   int y = static_cast<int>(big);
   std::cout << x << ' ' << y << '\n';
}
```

Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
3 -1294967296
```

int x = 3.7; stored 3, and converting 3,000,000,000 to int gave −1,294,967,296 (3,000,000,000 − 2^32). Neither was reported by the compiler with -Wall -Wextra. The same narrowing with braces, int x{3.7};, is rejected:

**C++ source code (file: Task04/Task04_COMPILEFAIL.cpp)**

```
// Task 4: brace initialisation rejects narrowing (expected to fail to compile)
int main()
{
   int x{3.7};
   return x;
}
```

**Compiler messages with -Wall -Wextra (compilation failed, as intended)**

```
Task04_COMPILEFAIL.cpp: In function 'int main()':
Task04_COMPILEFAIL.cpp:4:11: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to 'int' [-Wnarrowing]
    4 |     int x{3.7};
      |           ^~~
```

**static_cast:** it is needed when the conversion is intended and should be visible: for example static_cast<double>(a) / b to force floating-point division, static_cast<int>(zone) to get the number of an enum class value, and static_cast<Zone>(code) to turn a checked number into a zone. It does not make a wrong conversion safe, it only says that the programmer meant it.

4.7 Alternative concept: every conversion explicit

The program below is the same program with brace initialisation and static_cast used for every conversion:

**C++ source code (file: Task04/Task04_fixed.cpp)**

```
// Task 4 (alternative): every conversion written explicitly
#include <iostream>
int main()
{
   int a{7}, b{2};
   double c{static_cast<double>(a / b)};          // integer division is now visible
```

```
    double d{static_cast<double>(a) / b};
    std::cout << c << ' ' << d << '\n';
    char ch{static_cast<char>('A' + 2)};
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';
    bool flag{5 != 0};                          // int to bool written as a comparison
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
    unsigned int u{1};
    int s{-1};
    std::cout << (s < static_cast<int>(u) ? "s < u" : "s >= u") << '\n';
}
```

Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
3 3.5
C 68
1 true
s < u
```

The first three lines are the same as before. The last line changed from "s >= u" to "s < u". This is a real difference in behaviour, not only in style: the original program converted −1 to a huge unsigned number, while the explicit version converts u to int, so the comparison is −1 < 1. The explicit version states the intention in the source code, and the reader can now see that the comparison is signed. (An explicit cast in the other direction, static_cast<unsigned>(s), would reproduce the old surprise but would be visible.)

| Point | Implicit conversions (original) | Explicit casts and braces |
|---|---|---|
| Running time | O(1) per conversion | O(1). The same machine instructions are normally generated, so there is no change |
| Memory | No extra memory | No extra memory |
| Errors caught at compile time | Only some are warned (sign-compare) | Narrowing in brace initialisation is an error. The programmer must write each conversion, so accidental ones are easy to see in review |
| Errors still possible at run time | Wrong values after conversion | A cast can still hide a wrong conversion (for example casting a large long long to int). The cast silences the warning |
| Readability | Shorter code | Longer code but the intention is clear |

4.8 Conclusion

Four conversion surprises were demonstrated and explained (integer division, char promotion, int to bool, signed to unsigned). In the fare logic, money is long long throughout, all multiplications are done before divisions, and casts are written explicitly where an enum class or a double is involved.

Task 5 – Write Correct Expressions with the Right Precedence (4 Marks)

5.1 Requirement

Write the expressions for the peak check, the surcharge and the concession. Discuss precedence and associativity (including ?: and compound assignment), x++ against ++x and why i = i++

is a mistake, / and % with negative operands, and the two forms of the peak expression. Predict the output of the supplied program. The alternative concept is a lookup table bool peak[24].
5.2 Precedence and associativity of the operators used
The table lists the operators of this task from highest to lowest precedence (operators on the same row have the same precedence).

| Level | Operators | Associativity | Use in the fare logic |
|---|---|---|---|
| 1 (highest) | x++ x-- :: () [] static_cast | Left to right | BASE_PAISE[i], Zone::Inner, x++ |
| 2 | ++x --x ! unary − | Right to left | ++y, !peak |
| 3 | * / % | Left to right | f * (100 - pct) / 100, x % 4 * 2 |
| 4 | + − | Left to right | base + base * 20 / 100 |
| 5 | << >> | Left to right | std::cout << a << b |
| 6 | < <= > >= | Left to right | hour >= 8, hour < 10 |
| 7 | == != | Left to right | z == Zone::Inner |
| 8 | && | Left to right | (hour >= 8 && hour < 10) |
| 9 | \|\| | Left to right | (... ) \|\| (...) |
| 10 (lowest) | ?: = += -= *= /= | Right to left | peak ? a : b, balance -= f |

5.3 Expressions for the fare logic
```
// peak check (one half-open interval per peak period)
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);

// surcharge: multiply first, then divide, so the result stays exact in paise
if (peak) f += f * PEAK_SURCHARGE_PERCENT / 100;

// concession: f * (100 - pct) / 100   (NOT f * ((100 - pct) / 100), which gives 0)
f = f * (100 - concessionPercent) / 100;

// conditional operator for a simple choice
long long surcharge = peak ? f * PEAK_SURCHARGE_PERCENT / 100 : 0;
```

**Peak check.** && has higher precedence than ||, so hour >= 8 && hour < 10 || hour >= 17 && hour < 20 means the same as the version with brackets. The brackets are kept because the reader should not have to remember the precedence table. The relational operators >= and < bind tighter than &&, so no brackets are needed around them. Both are half-open intervals, so 8:00 and 17:00 are peak and 10:00 and 20:00 are not.
**Surcharge.** f += f * PEAK_SURCHARGE_PERCENT / 100 is evaluated as follows: * and / have the same level and group left to right, so (f * 20) / 100. The compound assignment has the lowest precedence, so the whole right side is calculated first and then added to f. For f = 3000: 3000 × 20 = 60000, 60000 / 100 = 600, f becomes 3600.
**Concession.** Brackets are needed around 100 - concessionPercent because - has lower precedence than *. The multiplication is done before the division so that no part is lost: 3600 × 50 / 100 = 1800.
**Conditional operator.** ?: has very low precedence, only above the comma operator, and associates right to left. Inside cout << it must be in brackets, as in (x > y ? x : y), because << has higher precedence than ?:.

**Compound assignment.** a += b * c means a = a + (b * c). Assignments associate right to left, so p = q = r assigns r to q first (shown in the program below).

5.4 x++ against ++x, and i = i++

x++ (post-increment) gives the old value of x and then increases x. ++x (pre-increment) increases x first and gives the new value. The statement i = i++; tries to change i twice in one expression: once by the increment and once by the assignment. Before C++17 the behaviour was undefined. In C++17 the order is defined and the assignment happens last, so i ends up with its old value and the increment is lost; the code is still a mistake because it does nothing useful. g++ warns about it. The next program shows the actual behaviour:

**C++ source code (file: Task05/Task05_incdemo.cpp)**

```
// Task 5: post-increment, pre-increment and i = i++
#include <iostream>
int main()
{
   int a = 5;
   int b = a++;   // b gets the old value, then a increases
   int c = 5;
   int d = ++c;   // c increases first, d gets the new value
   std::cout << "a=" << a << " b=" << b << " c=" << c << " d=" << d << '\n';

   int i = 5;
   i = i++;       // mistake: the increment is lost
   std::cout << "i after i = i++ : " << i << '\n';

   int j = 10;
   j += 3 * 2;    // compound assignment: right side evaluated first
   std::cout << "j after j += 3 * 2 : " << j << '\n';

   int p = 1, q = 2, r = 3;
   p = q = r;     // = is right-to-left
   std::cout << "p=" << p << " q=" << q << " r=" << r << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task05_incdemo Task05_incdemo.cpp

**Compiler messages with -Wall -Wextra**

```
Task05_incdemo.cpp: In function 'int main()':
Task05_incdemo.cpp:12:7: warning: operation on 'i' may be undefined [-Wsequence-point]
   12 |     i = i++;       // mistake: the increment is lost
      |     ~~^~~~
Task05_incdemo.cpp:12:7: warning: operation on 'i' may be undefined [-Wsequence-point]
```

**Actual output (from execution)**

```
a=6 b=5 c=6 d=6
i after i = i++ : 5
j after j += 3 * 2 : 16
p=3 q=3 r=3
```

b got the old value 5 and a became 6; d and c both became 6; i stayed at 5 after i = i++, and g++ gave the warning operation on 'i' may be undefined [-Wsequence-point]. j += 3 * 2 gave 16 (10 + 6), and p = q = r set all three to 3.

5.5 / and % with negative operands
Since C++11 integer division truncates towards zero, and the remainder has the sign of the left operand, so that (a / b) * b + a % b == a always holds. For example −10 / 3 is −3 and −10 % 3 is −1 (check: −3 × 3 + (−1) = −10). The program in 5.6 prints these values. This matters for the fare logic: percentages and times are always non-negative there, but a negative recharge or balance must be rejected before any / or % is used.
5.6 Predict the output
**My predicted output (written before running the program):**
11 16
9 6 4
6 4
3 1 -3 -1
18

**C++ source code (exactly as supplied) (file: Task05/Task05.cpp)**
```
#include <iostream>
int main()
{
   int x = 5, y = 3;
   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
   int z = x++ + ++y;
   std::cout << z << ' ' << x << ' ' << y << '\n';
   std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
   std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
   int k = 0;
   for (int i = 0; i < 10; i += 3) k += i;
   std::cout << k << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task05 Task05.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**
11 16
9 6 4
6 4
3 1 -3 -1
18

[Screenshot to be inserted: Task05_Output.png - code and output from the compiler window]
**Comparison with the prediction:** the predicted and the actual output are identical, and the compiler gave no warning.
5.7 Operator-by-operator explanation

| Output line | Operators and order of evaluation | Result |
|---|---|---|
| 11 16 | x + y * 2: * (level 3) before + (level 4): 3 × 2 = 6, then 5 + 6. (x + y) * 2: brackets first: 8, then × 2. Each expression is a separate operand of <<, which groups left to right | 11 and 16 |
| 9 6 4 | z = x++ + ++y: the postfix x++ gives 5 and x becomes 6; the prefix ++y makes y 4 and gives 4; + adds 5 + 4 = 9; = (lowest, right to left) stores 9 in z. Then z, x, y are printed | z = 9, x = 6, y = 4 |

| Output line | Operators and order of evaluation | Result |
|---|---|---|
| 6 4 | (x > y ? x : y): > first: 6 > 4 is true, so the result is x = 6 (the brackets are required inside <<). x % 4 * 2: % and * have the same level and group left to right, so (6 % 4) * 2 = 2 * 2. Without this rule x % (4 * 2) would be 6 | 6 and 4 |
| 3 1 -3 -1 | 10 / 3 = 3 and 10 % 3 = 1 (truncation towards zero). In -10 / 3 the unary minus (level 2) is applied to 10 first, then /: −10 / 3 = −3. −10 % 3 = −1 (sign of the left operand) | 3 1 -3 -1 |
| 18 | The loop variable i takes the values 0, 3, 6, 9 (i += 3 is a compound assignment). k += i adds each value: 0 + 3 + 6 + 9. At i = 12 the condition i < 10 fails | k = 18 |

5.8 Alternative concept: lookup table bool peak[24]
With a table indexed by the hour, the peak check becomes peak[hour]. The table is filled once at start-up (in this program from the expression itself, but it could be written as 24 constants):
**C++ source code (file: Task05/Task05_peaktable.cpp)**

```
// Task 5 (alternative): lookup table bool peak[24] against the && / || expression
#include <iostream>

bool peakExpr(int hour) {
   return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

int main() {
   bool peak[24] = {};
   for (int h = 0; h < 24; ++h) peak[h] = peakExpr(h);   // filled once at start-up

   bool same = true;
   int peakHours = 0;
   for (int h = 0; h < 24; ++h) {
      if (peak[h] != peakExpr(h)) same = false;
      if (peak[h]) ++peakHours;
   }
   std::cout << "table agrees with expression for all 24 hours: " << std::boolalpha << same <<
'\n';
   std::cout << "number of peak hours: " << peakHours << '\n';
   std::cout << "peak[8]=" << peak[8] << " peak[9]=" << peak[9] << " peak[10]=" << peak[10]
          << " peak[17]=" << peak[17] << " peak[19]=" << peak[19] << " peak[20]=" <<
peak[20] << '\n';
   std::cout << "sizeof(peak) = " << sizeof(peak) << " bytes\n";
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task05_peaktable Task05_peaktable.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**

```
table agrees with expression for all 24 hours: true
number of peak hours: 5
```

peak[8]=true peak[9]=true peak[10]=false peak[17]=true peak[19]=true peak[20]=false
sizeof(peak) = 24 bytes

| Point | Expression (&& and \|\|) | Lookup table peak[24] |
|---|---|---|
| Time per check | O(1). Between 2 and 4 comparisons depending on the hour (for example 3 for hour 12, 4 for hour 22) because && and \|\| stop early | O(1). One array access, but the hour must be checked to be 0–23 first or the index is out of range |
| Extra space | None | 24 bytes (sizeof(peak) = 24 in the output above) plus the code that fills it |
| Change of peak hours | Change two numbers in the expression (or constants) | Change the table contents. More error-prone because 24 flags must be kept consistent |
| Minutes (10:00 against 9:59) | Easy: use hour × 60 + minute (Task 6 and Task 9) | A table by hour cannot express it. A table by minute needs 1440 entries |
| Readability | The two intervals are visible in the code | The intervals are hidden in the data |

The program confirms that the table and the expression agree for all 24 hours (5 peak hours: 8, 9, 17, 18, 19). The time per check is O(1) in both designs; the table needs fewer comparisons but costs 24 bytes and a bounds check. I did not benchmark the difference because a few comparisons are far below the cost of the rest of a gate transaction.

5.9 Conclusion

The bracketed && / || expression is chosen for the peak check because it is O(1), needs no storage and can use minutes. Multiplication before division is used for the surcharge and the concession so every result is exact in paise. The predicted output matched the actual output for all five lines.

Task 6 – Design the Control Structures (4 Marks)

6.1 Requirement

Design the decision and looping logic for computing a fare and processing a day's journeys. Discuss if–else, switch and ?:, the three kinds of loops, break and continue, and a switch with no matching case. Write pseudocode before the C++ code, complete TODO 1 and TODO 2, execute the program, and test 10 AM and 8 PM. The alternative concept is an array indexed by the zone.

6.2 Choice of control structures

**if–else** is used for conditions that are true or false: the peak check, the minimum balance check, and the surcharge. It can express ranges (hour >= 8 && hour < 10), which a switch cannot.

**switch** is used for the zone, because the zone has a small fixed set of values and each value gives a different base fare. It is clearer than a chain of if (z == ...) and the compiler can warn when an enumerator is missing (-Wswitch).

**Conditional operator ?:** is used for a short choice that returns a value, for example peak ? "peak" : "off-peak" in the output. It is not used for the fare steps because nested ?: is hard to read.

**for loop:** best when the index is needed or the number of repetitions is known, for example the 24 hours in Task 5.

**Range-based for loop:** best for processing the day's journeys stored in an array or vector (for (const Trip& t : trips)). It has no index, so there is no off-by-one error, and const stops accidental changes. It is the best choice here and is used in the supplied program.
**while loop:** best when the number of repetitions is not known, for example reading journeys until the input ends (while (std::cin >> zone >> hour)), used in Task 8.
**break and continue:** break is needed at the end of every case, otherwise execution falls through into the next case. continue is used to skip an invalid journey (unknown zone or low balance) and go on with the next one, so one bad record does not stop the day's processing. break can also be used to stop the loop if the gate must shut down.
**A switch with no matching case:** nothing inside the switch is executed and the program continues after it. If there is no default, the variable that should have been set keeps its old value and a function with no return has undefined behaviour. Therefore the function below has a default that returns −1, which means "no valid fare", and canTravel() refuses a negative fare.

6.3 Pseudocode

The pseudocode follows the common ALGORITHM / INPUT / OUTPUT / BEGIN–END format. [Check keywords and layout against the SIMATS Engineering Standard for Algorithmic Pseudocode given by the faculty and adjust if needed]

```
ALGORITHM ComputeFare
INPUT   : zone, peak (TRUE or FALSE)
OUTPUT  : fare in paise, or -1 if the zone is unknown
BEGIN
   base <- 0
   SELECT zone
      CASE Inner  : base <- BASE_PAISE[0]
      CASE Middle : base <- BASE_PAISE[1]
      CASE Outer  : base <- BASE_PAISE[2]
      OTHERWISE   : RETURN -1
   END SELECT
   IF peak = TRUE THEN
      base <- base + (base * PEAK_SURCHARGE_PERCENT) DIV 100
   END IF
   RETURN base
END

ALGORITHM CanTravel
INPUT   : balance, fare (both in paise)
OUTPUT  : TRUE or FALSE
BEGIN
   IF fare < 0 THEN RETURN FALSE
   RETURN (balance - fare) >= MIN_BALANCE_PAISE
END

ALGORITHM ProcessDay
INPUT   : balance, list of trips (zone, peak)
OUTPUT  : message for each trip and the final balance
BEGIN
   FOR EACH trip IN list DO
      f <- ComputeFare(trip.zone, trip.peak)
      IF CanTravel(balance, f) THEN
         balance <- balance - f
         PRINT fare and new balance
```

```
      ELSE
         PRINT "Denied (low balance)" and unchanged balance
      END IF
   END FOR
END
```

6.4 Complete the code: TODO 1 and TODO 2

Only the bodies of fare() and canTravel() were written. The rest of the supplied program (headers, enum, constants, main()) is unchanged; only the indentation was restored because it was lost in the document. The default branch and the f >= 0 test are small safety additions for unknown zones.

**Completed C++ source code (file: Task06/Task06.cpp)**

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
   // TODO 1: switch on the zone, then add the surcharge with integer arithmetic only
   long long base = 0;
   switch (z) {
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
      default: return -1;   // unknown zone: no valid fare
   }
   if (peak) {
      base += base * PEAK_SURCHARGE_PERCENT / 100;
   }
   return base;
}

bool canTravel(long long balance, long long f)
{
   // TODO 2: the balance left after paying must be at least MIN_BALANCE_PAISE
   return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
   long long balance = 10000; // Rs 100.00 in paise
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
   for (const Trip& t : trips) {
      long long f = fare(t.zone, t.peak);
      if (canTravel(balance, f)) {
```

```
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task06 Task06.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Expected output given in the assignment:**
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
**Actual output (from execution)**
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

[Screenshot to be inserted: Task06_Output.png - code and output from the compiler window]
The actual output is identical to the expected output. **Check by hand:** trip 1 is Inner off-peak, 1000 paise, 10000 − 1000 = 9000 ≥ 5000, allowed, balance Rs 90. Trip 2 is Outer peak: 3000 + 3000 × 20 / 100 = 3000 + 600 = 3600 paise, 9000 − 3600 = 5400 ≥ 5000, allowed, balance Rs 54. Trip 3 is Middle off-peak: 2000 paise, 5400 − 2000 = 3400 < 5000, denied, balance unchanged. The fare is printed as f / 100 rupees, a dot, and f % 100 padded to two digits with setw(2) and setfill('0'), which is why 3600 paise prints as 36.00. setfill(' ') restores the fill character after use.
6.5 Boundary test: exactly 10:00 AM and 8:00 PM
The supplied fare() takes a ready-made bool peak, so the peak rule is tested with a separate function isPeak(hour, minute) using minutes from midnight. The test program (kept in a separate file so that Task06.cpp stays as supplied) uses an Outer zone journey with Rs 200 on the card.
**C++ source code (file: Task06/Task06_boundary.cpp)**

```
// Task 6: boundary check at 10:00 AM and 8:00 PM (functions as in Task06.cpp)
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
```

```
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }
    return base;
}

bool canTravel(long long balance, long long f)
{
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

// peak: 8:00 <= time < 10:00 or 17:00 <= time < 20:00
bool isPeak(int hour, int minute)
{
    int m = hour * 60 + minute;
    return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
}

int main()
{
    long long balance = 20000; // Rs 200.00
    struct Trip { int hour; int minute; };
    Trip trips[] = {{7, 59}, {8, 0}, {9, 59}, {10, 0}, {16, 59}, {17, 0}, {19, 59}, {20, 0}};
    for (const Trip& t : trips) {
        bool peak = isPeak(t.hour, t.minute);
        long long f = fare(Zone::Outer, peak);
        std::cout << std::setw(2) << std::setfill('0') << t.hour << ':' << std::setw(2) << t.minute
                  << std::setfill(' ') << (peak ? "  peak     " : "  off-peak ")
                  << "Outer fare Rs " << f / 100 << '.' << std::setw(2) << std::setfill('0')
                  << f % 100 << std::setfill(' ')
                  << "  allowed: " << std::boolalpha << canTravel(balance, f) << '\n';
    }
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task06_boundary Task06_boundary.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
07:59  off-peak Outer fare Rs 30.00  allowed: true
08:00  peak     Outer fare Rs 36.00  allowed: true
09:59  peak     Outer fare Rs 36.00  allowed: true
10:00  off-peak Outer fare Rs 30.00  allowed: true
16:59  off-peak Outer fare Rs 30.00  allowed: true
17:00  peak     Outer fare Rs 36.00  allowed: true
19:59  peak     Outer fare Rs 36.00  allowed: true
20:00  off-peak Outer fare Rs 30.00  allowed: true
```

10:00 is off-peak (Rs 30.00), 9:59 is peak (Rs 36.00), 20:00 is off-peak (Rs 30.00) and 19:59 is peak (Rs 36.00). The opening times 8:00 and 17:00 are peak, and 7:59 and 16:59 are off-peak. This matches the half-open rule and removes the inconsistency of the first version.

6.6 Alternative concept: array indexed by zone

Here the switch is replaced by constexpr long long FARE[] = {1000, 2000, 3000, 6000}; indexed with the zone. The extra Airport entry is used to show how a fourth zone is added.

**C++ source code (file: Task06/Task06_array.cpp)**

```
// Task 6 (alternative): fare looked up from an array indexed by the zone
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer, Airport };

constexpr long long FARE[] = {1000, 2000, 3000, 6000};
constexpr int ZONE_COUNT = sizeof(FARE) / sizeof(FARE[0]);
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
   int i = static_cast<int>(z);
   if (i < 0 || i >= ZONE_COUNT) return -1;     // range check replaces the switch default
   long long base = FARE[i];
   if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
   return base;
}

bool canTravel(long long balance, long long f)
{
   return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
   long long balance = 10000;
   struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
   for (const Trip& t : trips) {
      long long f = fare(t.zone, t.peak);
      if (canTravel(balance, f)) {
         balance -= f;
         std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                   << std::setfill('0') << f % 100 << std::setfill(' ');
      } else {
         std::cout << "Denied (low balance)";
      }
      std::cout << " balance Rs " << balance / 100 << '\n';
   }
   std::cout << "Airport peak fare: " << fare(Zone::Airport, true) << " paise\n";
   std::cout << "Invalid zone fare: " << fare(static_cast<Zone>(9), false) << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task06_array Task06_array.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
Airport peak fare: 7200 paise
Invalid zone fare: -1

| Point | switch on zone | array FARE[zone] |
|---|---|---|
| Time per lookup | O(1). The compiler may build a jump table or a short chain of comparisons; for three or four cases both are tiny | O(1). One indexed load |
| Space | No data array. The values are in the instructions or in BASE_PAISE | The array: 3 × 8 = 24 bytes for three zones, 4 × 8 = 32 bytes for four |
| Fourth zone | Add an enumerator and a new case. If a case is forgotten, -Wswitch warns only when there is no default; with a default the compiler stays silent and the zone silently gets −1 (shown in Task 12) | Add an enumerator and one value. If the array is shorter than the enum, the index is out of range, which is undefined behaviour, so a range check (as in the program) or static_assert is needed |
| Special rules per zone | Easy: each case can contain different code | Not possible, it only holds data |
| Readability and maintenance | More lines, but each zone is visible | Less code, and the fares are all in one table |

The output is identical for the three original trips, and the fourth zone (Airport peak = 6000 + 1200 = 7200 paise) works by adding one table entry. For plain data such as a fare per zone the array is the better design; the switch is better when zones need different logic.
6.7 Conclusion
A switch on the zone with integer paise, a plain if for the surcharge and the balance check, and a range-based for for the day's journeys were chosen. TODO 1 and TODO 2 were completed, the output matched the expected output exactly, and the 10:00 and 20:00 boundaries were verified.
Task 7 – From Procedural to Object-Oriented (4 Marks)
7.1 Requirement
Write the fare logic in two styles: procedural with free functions, and object-oriented with a SmartCard class that keeps its balance private and offers deduct() and canTravel(). Explain how the class prevents a negative balance and which version is easier to change when the minimum balance changes. The alternative concept is a public balance member.
7.2 Procedural version
**C++ source code (file: Task07/Task07_Procedural.cpp)**

```
// Task 7: procedural version - separate variables and free functions
#include <iostream>

constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
long long balancePaise = 10000;   // global: any function can change it

bool canTravel(long long fare)
{
   return fare >= 0 && balancePaise - fare >= MIN_BALANCE_PAISE;
}

bool deduct(long long fare)
{
   if (!canTravel(fare)) return false;
   balancePaise -= fare;
   return true;
}

int main()
{
   std::cout << "Start balance: " << balancePaise << " paise\n";
   std::cout << "Deduct 3600: " << std::boolalpha << deduct(3600) << ", balance " <<
balancePaise << '\n';
   std::cout << "Deduct 3600: " << deduct(3600) << ", balance " << balancePaise << '\n';

   // nothing stops a careless line like this one in some other part of the program
   balancePaise -= 9000;
   std::cout << "After direct change: balance " << balancePaise << " paise\n";
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task07_Procedural Task07_Procedural.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
Start balance: 10000 paise
Deduct 3600: true, balance 6400
Deduct 3600: false, balance 6400
After direct change: balance -2600 paise
```

The functions canTravel() and deduct() work correctly. The second deduction is refused (6400 − 3600 = 2800 < 5000). But the global balancePaise is open to every part of the program, and the line balancePaise -= 9000; (which could be written by anyone, anywhere) made the balance −2600. This is the same kind of fault as the negative balance in the first version.

7.3 Object-oriented version

**C++ source code (file: Task07/Task07_OOP.cpp)**

```
// Task 7: object-oriented version - SmartCard keeps its balance private
#include <iostream>

constexpr long long MIN_BALANCE_PAISE = 5000;

class SmartCard {
public:
   explicit SmartCard(long long initialPaise) : balancePaise(initialPaise < 0 ? 0 : initialPaise)
{}
```

```
    bool canTravel(long long fare) const
    {
        return fare >= 0 && balancePaise - fare >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long fare)
    {
        if (!canTravel(fare)) return false;   // the only way to reduce the balance
        balancePaise -= fare;
        return true;
    }

    bool recharge(long long amountPaise)
    {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }

    long long balance() const { return balancePaise; }

private:
    long long balancePaise;
};

int main()
{
    SmartCard card(10000);
    std::cout << std::boolalpha;
    std::cout << "Start balance: " << card.balance() << " paise\n";
    std::cout << "Deduct 3600: " << card.deduct(3600) << ", balance " << card.balance() << '\n';
    std::cout << "Deduct 3600: " << card.deduct(3600) << ", balance " << card.balance() << '\n';
    std::cout << "Deduct -500: " << card.deduct(-500) << ", balance " << card.balance() << '\n';
    std::cout << "Recharge -100: " << card.recharge(-100) << ", balance " << card.balance() <<
'\n';
    std::cout << "Recharge 2000: " << card.recharge(2000) << ", balance " << card.balance() <<
'\n';
    // card.balancePaise -= 9000;   // would not compile: balancePaise is private
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task07_OOP Task07_OOP.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**

```
Start balance: 10000 paise
Deduct 3600: true, balance 6400
Deduct 3600: false, balance 6400
Deduct -500: false, balance 6400
Recharge -100: false, balance 6400
Recharge 2000: true, balance 8400
```

[Screenshot to be inserted: Task07_Output.png - code and output from the compiler window]
The same two deductions behave the same way (the second is refused), a negative fare is refused too, and a negative recharge is refused. The balance changes only by an allowed deduct() or by a positive recharge(2000) (6400 to 8400).
Demonstration that the private balance cannot be changed from outside
**C++ source code (file: Task07/Task07_COMPILEFAIL.cpp)**

```
// Task 7: the private balance cannot be changed from outside (expected to fail to compile)
class SmartCard {
public:
   explicit SmartCard(long long p) : balancePaise(p) {}
   long long balance() const { return balancePaise; }
private:
   long long balancePaise;
};

int main()
{
   SmartCard card(10000);
   card.balancePaise -= 9000;   // error: balancePaise is private
   return 0;
}
```

**Compiler messages with -Wall -Wextra (compilation failed, as intended)**

```
Task07_COMPILEFAIL.cpp: In function 'int main()':
Task07_COMPILEFAIL.cpp:13:10: error: 'long long int SmartCard::balancePaise' is private within this context
  13 |    card.balancePaise -= 9000;   // error: balancePaise is private
     |         ^~~~~~~~~~~~
Task07_COMPILEFAIL.cpp:7:15: note: declared private here
   7 |    long long balancePaise;
     |              ^~~~~~~~~~~~
Task07_COMPILEFAIL.cpp:13:10: note: field 'long long int SmartCard::balancePaise' can be accessed via 'long long int SmartCard::balance() const'
  13 |    card.balancePaise -= 9000;   // error: balancePaise is private
     |         ^~~~~~~~~~~~
     |         balance()
```

7.4 How the class prevents a negative balance
balancePaise is private, so no code outside the class can write to it (the compiler error above). deduct() is the only way to reduce the balance and it calls canTravel() first. Since canTravel() requires balance − fare >= 5000, the new balance is at least 5000, so it can never be negative. canTravel() and deduct() refuse a negative fare, which would otherwise increase the balance without a recharge (output: Deduct −500 gives false).
recharge() refuses zero and negative amounts (output: Recharge −100 gives false).
The constructor sets a negative starting balance to 0.
7.5 Which version is easier to change when the minimum balance changes?
If only the number changes (for example Rs 50 to Rs 75), both versions need one edit, because MIN_BALANCE_PAISE is a constant used in one function. The difference appears when the rule changes in shape, for example when the minimum balance depends on the type of card or on the zone. In the object-oriented version the change is made inside SmartCard::canTravel() and every caller gets it automatically, because nobody else touches the balance. In the

procedural version every function and every line that reads or changes the global balance has to be searched and checked, and as the test showed, a direct change can exist anywhere. The object-oriented version is therefore easier and safer to change.

| Point | Procedural | Object-oriented |
|---|---|---|
| Time per journey | O(1) | O(1). A member function call has the same cost as a free function call |
| Memory per card | One global long long, or an array of balances | One long long member (8 bytes) |
| Who can change the balance | Any code | Only member functions |
| Rule change in the minimum balance logic | Search the whole program | One member function |
| Amount of code | Slightly shorter | Slightly longer (class, constructor, access functions) |
| Testing | Global state must be reset between tests | Each test creates its own object |

7.6 Alternative concept: public data member

**C++ source code (file: Task07/Task07_PublicMember.cpp)**

```
// Task 7 (alternative): balance kept in a public data member
#include <iostream>

struct PublicCard {
   long long balance;   // public: any code can change it
};

int main()
{
   PublicCard card{10000};
   card.balance -= 3600;              // no check at all
   card.balance -= 3600;              // balance is now below Rs 50
   card.balance -= 9000;              // and now it is negative
   std::cout << "Balance: " << card.balance << " paise\n";
   std::cout << "Rule 'at least 5000 paise must remain' broken: "
          << std::boolalpha << (card.balance < 5000) << '\n';
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task07_PublicMember Task07_PublicMember.cpp

Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
Balance: -6200 paise
Rule 'at least 5000 paise must remain' broken: true
```

With a public balance the time and the space do not change: a subtraction is still O(1) and the member is still one long long (8 bytes). What changes is what any part of the program can do. The output shows the balance −6200 and that the rule "at least 5000 paise must remain" is

broken. The rules that anyone can now break are: R1 (minimum balance), R2 (never negative), R8 (recharge must be positive, because card.balance += -500; is possible) and R9 (the check before deduction can be skipped). Also, a later change in how the balance is stored (for example storing it in a database) would force every line that uses the member to be rewritten.

7.7 Conclusion

The object-oriented version is chosen. It gives the same speed and memory as the procedural version, and it makes the minimum balance and non-negative balance rules impossible to bypass.

Task 8 – Read and Print Data Safely (4 Marks)

8.1 Requirement

Design the input and output of the gate program: read values with cin and check cin.fail() when letters are typed, clear the error state with cin.clear() and discard the bad input, format money with <iomanip>, and print a neat journey report. The alternative concept is reading each line with getline() and converting with stoi() inside try/catch.

8.2 How the input handling works

When cin >> number meets letters, nothing is read, the stream goes into a fail state (cin.fail() becomes true) and every later read does nothing until the state is cleared. A program that does not handle this loops forever or uses the old value.

cin.clear() resets the error flags. It does not remove the bad characters, so cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n') is also needed to throw away the rest of the line. Without the ignore, the next read fails on the same letters again.

After a successful read the rest of the line is also discarded, so leftover characters (for example the x in 7x) do not disturb the next question.

If the input ends (end of file), cin.eof() is true. The function then returns false, because otherwise the retry loop would never finish.

A valid number is also checked for range (recharge 1–2000, zone 0–2, hour 0–23). A zone code such as 5 would give an out-of-range array index, and a negative recharge would reduce the balance.

8.3 Formatting money with iomanip

std::setw(n) sets the width of the next output only. std::setfill(c) sets the fill character and stays active until changed, so it is reset with setfill(' ') afterwards.

std::fixed with std::setprecision(2) prints a double with exactly two digits after the decimal point (36 prints as 36.00). Both stay active for later output.

Because money is stored in paise, the exact method is used: fare / 100, a dot, then fare % 100 with setw(2) and setfill('0'). This avoids any double in the money value. The double method (fare / 100.0) is used only in the report lines to show right-aligned columns.

The receipt number uses setw(6) and setfill('0') to print 000042.

8.4 Program using cin

To demonstrate the recovery, the program was run with this input file (input.txt), which contains mistakes on purpose, one line for each answer:

**Input file used for both programs (input.txt)**

```
abc
-50
200
5
two
2
nine
7x
17
```

**C++ source code (file: Task08/Task08_cin.cpp)**

```
// Task 8: safe input with cin, cin.fail(), cin.clear() and ignore()
#include <iomanip>
#include <iostream>
#include <limits>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;

// returns false only when the input ends
bool readInt(const char* prompt, int low, int high, int& value)
{
   while (true) {
      std::cout << prompt;
      if (std::cin >> value) {
         // discard whatever is left on the line (for example the "x" in "7x")
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
         if (value >= low && value <= high) return true;
         std::cout << "  Out of range (" << low << " to " << high << "), try again.\n";
      } else {
         if (std::cin.eof()) return false;
         std::cin.clear();                                        // reset error state
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  // discard bad input
         std::cout << "  Not a number, try again.\n";
      }
   }
}

int main()
{
   int rupees = 0, zoneCode = 0, hour = 0;
   if (!readInt("Recharge amount in rupees (1-2000): ", 1, 2000, rupees)) return 1;
   if (!readInt("Zone (0 = Inner, 1 = Middle, 2 = Outer): ", 0, 2, zoneCode)) return 1;
   if (!readInt("Hour of travel (0-23): ", 0, 23, hour)) return 1;

   long long balance = rupees * 100LL;
   bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
   long long fare = BASE_PAISE[zoneCode];
   if (peak) fare += fare * PEAK_SURCHARGE_PERCENT / 100;
   balance -= fare;

   const char* zoneName[] = {"Inner", "Middle", "Outer"};
   std::cout << "\n===== Journey Report =====\n";
   std::cout << "Receipt no : " << std::setw(6) << std::setfill('0') << 42 << std::setfill(' ') << '\n';
   std::cout << "Zone       : " << zoneName[zoneCode] << '\n';
   std::cout << "Time       : " << std::setw(2) << std::setfill('0') << hour << ":00"
             << std::setfill(' ') << (peak ? " (peak)" : " (off-peak)") << '\n';
   std::cout << "Fare       : Rs " << std::setw(8) << std::fixed << std::setprecision(2)
             << fare / 100.0 << '\n';
```

```
    std::cout << "Balance    : Rs " << std::setw(8) << balance / 100.0 << '\n';
    std::cout << "Fare (integer method): Rs " << fare / 100 << '.' << std::setw(2)
              << std::setfill('0') << fare % 100 << std::setfill(' ') << '\n';
}
```

Compile and run: g++ -std=c++17 -Wall -Wextra -o Task08_cin Task08_cin.cpp and ./Task08_cin < input.txt.
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
Recharge amount in rupees (1-2000):   Not a number, try again.
Recharge amount in rupees (1-2000):   Out of range (1 to 2000), try again.
Recharge amount in rupees (1-2000): Zone (0 = Inner, 1 = Middle, 2 = Outer):   Out of range (0 to 2), try again.
Zone (0 = Inner, 1 = Middle, 2 = Outer):   Not a number, try again.
Zone (0 = Inner, 1 = Middle, 2 = Outer): Hour of travel (0-23):   Not a number, try again.
Hour of travel (0-23):
===== Journey Report =====
Receipt no : 000042
Zone       : Outer
Time       : 07:00 (off-peak)
Fare       : Rs    30.00
Balance    : Rs   170.00
Fare (integer method): Rs 30.00
```

[Screenshot to be inserted: Task08_Output.png - code and output from the compiler window]
The program output does not show the typed text because the input came from a file, so every message follows its prompt on the same line. Step by step: abc for the recharge gave "Not a number" (fail, clear, ignore); -50 was a number but out of range; 200 was accepted. For the zone, 5 was out of range, two was not a number, 2 was accepted. For the hour, nine was not a number and then **`7x` was accepted as 7**, because cin >> int reads the digits and stops at the x, and the rest of the line is discarded by the ignore. So the journey was Outer zone at 07:00, off-peak: fare Rs 30.00 and balance 20000 − 3000 = 17000 paise = Rs 170.00. The double method and the integer method print the same fare, Rs 30.00.

8.5 Alternative concept: getline() and stoi() with try/catch

**C++ source code (file: Task08/Task08_getline.cpp)**

```
// Task 8 (alternative): getline() + stoi() inside try/catch
#include <iomanip>
#include <iostream>
#include <string>

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;

// returns false only when the input ends
bool readInt(const char* prompt, int low, int high, int& value)
{
    std::string line;
    while (true) {
        std::cout << prompt;
        if (!std::getline(std::cin, line)) return false;
        try {
```

```
            std::size_t used = 0;
            int v = std::stoi(line, &used);
            if (used != line.size()) throw std::invalid_argument("extra characters");
            if (v < low || v > high) {
                std::cout << "  Out of range (" << low << " to " << high << "), try again.\n";
                continue;
            }
            value = v;
            return true;
        } catch (const std::invalid_argument&) {
            std::cout << "  Not a valid whole number, try again.\n";
        } catch (const std::out_of_range&) {
            std::cout << "  Number too large, try again.\n";
        }
    }
}

int main()
{
    int rupees = 0, zoneCode = 0, hour = 0;
    if (!readInt("Recharge amount in rupees (1-2000): ", 1, 2000, rupees)) return 1;
    if (!readInt("Zone (0 = Inner, 1 = Middle, 2 = Outer): ", 0, 2, zoneCode)) return 1;
    if (!readInt("Hour of travel (0-23): ", 0, 23, hour)) return 1;

    long long balance = rupees * 100LL;
    bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    long long fare = BASE_PAISE[zoneCode];
    if (peak) fare += fare * PEAK_SURCHARGE_PERCENT / 100;
    balance -= fare;

    std::cout << "\nFare Rs " << fare / 100 << '.' << std::setw(2) << std::setfill('0') << fare %
100
              << std::setfill(' ') << ", balance Rs " << balance / 100 << '.' << std::setw(2)
              << std::setfill('0') << balance % 100 << std::setfill(' ') << '\n';
}
```

Compile and run: g++ -std=c++17 -Wall -Wextra -o Task08_getline Task08_getline.cpp and ./Task08_getline < input.txt.
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
**Actual output (from execution)**

```
Recharge amount in rupees (1-2000):   Not a valid whole number, try again.
Recharge amount in rupees (1-2000):   Out of range (1 to 2000), try again.
Recharge amount in rupees (1-2000): Zone (0 = Inner, 1 = Middle, 2 = Outer):   Out of range
(0 to 2), try again.
Zone (0 = Inner, 1 = Middle, 2 = Outer):   Not a valid whole number, try again.
Zone (0 = Inner, 1 = Middle, 2 = Outer): Hour of travel (0-23):   Not a valid whole number, try
again.
Hour of travel (0-23):   Not a valid whole number, try again.
Hour of travel (0-23):
Fare Rs 36.00, balance Rs 164.00
```

With the same input file, the result is different at the hour: 7x was rejected ("Not a valid whole number") because the program checks that stoi used the whole line (used != line.size()), and the next line 17 was accepted. The journey is therefore 17:00, which is peak: fare 3000 + 600 = 3600 paise (Rs 36.00) and balance Rs 164.00. This shows that the same input can give different journeys in the two designs.

| **Point** | **cin >> number** | **getline() + stoi() with try/catch** |
|---|---|---|
| Time for a line of length L | O(L): the digits are read one by one | O(L): the line is read once and converted once |
| Buffer space | The stream's own buffer, no extra string | A std::string of up to L characters, O(L) |
| Error handling | Error flags; clear() and ignore() are needed, and forgetting one gives an endless loop | No stream error for a bad number. stoi throws invalid_argument or out_of_range, which are caught, and the whole line is already consumed |
| Input like 7x or 9.5 | Accepts the leading 7 or 9 silently and leaves the rest for the next read (or discards it) | Can reject it, by comparing the used length with the line length |
| Number too large for int | Sets the failbit | Throws out_of_range |
| Simplicity | Shorter for simple input | More lines, but easier to validate completely |

8.6 Conclusion

Both designs recovered from letters and from out-of-range values and printed correctly formatted amounts. The cin version needs clear() and ignore() and can accept 7x silently. The getline() + stoi() version validates the whole line, so it is chosen for the gate program when strict input checking is needed.

Task 9 – Implement and Test the Fare Logic (4 Marks)

9.1 Requirement

Implement the complete fare logic and test: normal journeys in each zone (peak and off-peak), boundary cases (8:00, 10:00, 17:00, 20:00 and a balance leaving exactly Rs 50), invalid cases (unknown zone, negative recharge, letters for the hour), and a set of 5 journeys with a hand-calculated total. The alternative concept is a table of test cases checked in a loop.

9.2 Design of the program

Constants: base fares, surcharge, concessions and minimum balance are constexpr values at the top. Concessions are assumed (student 50%, senior 40%) because the assignment gives no figures.

Time is checked with minutes from midnight: peak if 480 ≤ minutes < 600 or 1020 ≤ minutes < 1200.

zoneFromCode() accepts only 0, 1 and 2, so an unknown zone code is rejected and the fare is −1.

SmartCard has a private balance, canTravel(), deduct() and recharge() (recharge accepts only positive amounts).

readHour() reads from a stream, and rejects letters, empty input and hours outside 0–23. The hour tests use std::istringstream so the "typed" input is repeatable; this is the same checking as the cin version in Task 8.

The test cases are a table of structures with the expected value written by hand (in paise). The program compares the expected value with the value calculated by the functions and prints PASS or FAIL. The program returns exit code 0 only if every test passes.

9.3 Source code

**C++ source code (file: Task09/Task09.cpp)**

```
// Task 9: complete fare logic with a table of test cases
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>

enum class Zone { Inner, Middle, Outer };
enum class Category { Regular, Student, Senior };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int ZONE_COUNT = 3;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_CONCESSION_PERCENT = 50;    // assumed, not given in the assignment
constexpr int SENIOR_CONCESSION_PERCENT = 40;     // assumed, not given in the assignment
constexpr long long MIN_BALANCE_PAISE = 5000;

bool zoneFromCode(int code, Zone& z)
{
   if (code < 0 || code >= ZONE_COUNT) return false;
   z = static_cast<Zone>(code);
   return true;
}

bool isPeak(int hour, int minute)
{
   int m = hour * 60 + minute;
   return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
}

long long fare(Zone z, bool peak, Category c)
{
   long long f = BASE_PAISE[static_cast<int>(z)];
   if (peak) f += f * PEAK_SURCHARGE_PERCENT / 100;
   if (c == Category::Student) f = f * (100 - STUDENT_CONCESSION_PERCENT) / 100;
   if (c == Category::Senior) f = f * (100 - SENIOR_CONCESSION_PERCENT) / 100;
   return f;
}

class SmartCard {
public:
   explicit SmartCard(long long paise) : balancePaise(paise) {}
   bool canTravel(long long f) const { return f >= 0 && balancePaise - f >= MIN_BALANCE_PAISE; }
   bool deduct(long long f)
```

```
    {
        if (!canTravel(f)) return false;
        balancePaise -= f;
        return true;
    }
    bool recharge(long long amount)
    {
        if (amount <= 0) return false;
        balancePaise += amount;
        return true;
    }
    long long balance() const { return balancePaise; }
private:
    long long balancePaise;
};

// reads an hour from a stream; false for letters, empty input or a value outside 0-23
bool readHour(std::istream& in, int& hour)
{
    int h;
    if (!(in >> h)) {
        in.clear();
        in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return false;
    }
    if (h < 0 || h > 23) return false;
    hour = h;
    return true;
}

int passed = 0, failed = 0;

void report(const std::string& id, const std::string& name, const std::string& expected,
            const std::string& actual)
{
    bool ok = (expected == actual);
    if (ok) ++passed; else ++failed;
    std::cout << std::left << std::setw(5) << id << std::setw(47) << name
              << "expected " << std::setw(10) << expected << "actual " << std::setw(10) << actual
              << (ok ? "PASS" : "FAIL") << '\n';
}

int main()
{
    // ---- Part A: fare tests (expected values worked out by hand, in paise; -1 means rejected) --
--
    struct FareCase { const char* id; const char* name; int zoneCode; int hour; int minute;
                      Category cat; long long expected; };
    const FareCase fareCases[] = {
        {"F01", "Inner off-peak 12:00", 0, 12, 0, Category::Regular, 1000},
        {"F02", "Inner peak 09:00", 0, 9, 0, Category::Regular, 1200},
```

```
        {"F03", "Middle off-peak 14:00", 1, 14, 0, Category::Regular, 2000},
        {"F04", "Middle peak 18:30", 1, 18, 30, Category::Regular, 2400},
        {"F05", "Outer off-peak 22:00", 2, 22, 0, Category::Regular, 3000},
        {"F06", "Outer peak 17:30", 2, 17, 30, Category::Regular, 3600},
        {"F07", "Boundary 07:59 (off-peak)", 0, 7, 59, Category::Regular, 1000},
        {"F08", "Boundary 08:00 (peak)", 0, 8, 0, Category::Regular, 1200},
        {"F09", "Boundary 09:59 (peak)", 0, 9, 59, Category::Regular, 1200},
        {"F10", "Boundary 10:00 (off-peak)", 0, 10, 0, Category::Regular, 1000},
        {"F11", "Boundary 16:59 (off-peak)", 0, 16, 59, Category::Regular, 1000},
        {"F12", "Boundary 17:00 (peak)", 0, 17, 0, Category::Regular, 1200},
        {"F13", "Boundary 19:59 (peak)", 0, 19, 59, Category::Regular, 1200},
        {"F14", "Boundary 20:00 (off-peak)", 0, 20, 0, Category::Regular, 1000},
        {"F15", "Student Outer off-peak", 2, 12, 0, Category::Student, 1500},
        {"F16", "Student Outer peak", 2, 18, 0, Category::Student, 1800},
        {"F17", "Senior Middle peak", 1, 9, 0, Category::Senior, 1440},
        {"F18", "Unknown zone code 5", 5, 12, 0, Category::Regular, -1},
        {"F19", "Unknown zone code -1", -1, 12, 0, Category::Regular, -1},
    };
    std::cout << "Part A: fare tests\n";
    for (const FareCase& t : fareCases) {
        Zone z;
        long long actual = -1;
        if (zoneFromCode(t.zoneCode, z)) actual = fare(z, isPeak(t.hour, t.minute), t.cat);
        report(t.id, t.name, std::to_string(t.expected), std::to_string(actual));
    }

    // ---- Part B: minimum balance tests ----
    struct BalCase { const char* id; const char* name; long long balance; long long fare; bool
expected; };
    const BalCase balCases[] = {
        {"B01", "Balance 6000, fare 1000 (leaves exactly 5000)", 6000, 1000, true},
        {"B02", "Balance 5999, fare 1000 (leaves 4999)", 5999, 1000, false},
        {"B03", "Balance 6000, fare 1001 (leaves 4999)", 6000, 1001, false},
        {"B04", "Balance 10000, fare 3600 (leaves 6400)", 10000, 3600, true},
        {"B05", "Balance 8000, fare 3600 (leaves 4400)", 8000, 3600, false},
    };
    std::cout << "\nPart B: minimum balance tests\n";
    for (const BalCase& t : balCases) {
        SmartCard card(t.balance);
        report(t.id, t.name, t.expected ? "allowed" : "denied", card.canTravel(t.fare) ? "allowed" :
"denied");
    }

    // ---- Part C: invalid input tests ----
    std::cout << "\nPart C: invalid input tests\n";
    {
        SmartCard card(10000);
        report("R01", "Recharge -500", "rejected", card.recharge(-500) ? "accepted" : "rejected");
        report("R02", "Balance unchanged after bad recharge", "10000",
std::to_string(card.balance()));
        report("R03", "Recharge 0", "rejected", card.recharge(0) ? "accepted" : "rejected");
```

```
    report("R04", "Recharge 5000", "accepted", card.recharge(5000) ? "accepted" :
"rejected");
    report("R05", "Balance after valid recharge", "15000", std::to_string(card.balance()));
  }
  const char* hourInputs[] = {"abc", "", "25", "-1", "9"};
  const char* hourExpected[] = {"rejected", "rejected", "rejected", "rejected", "accepted"};
  const char* hourNames[] = {"Hour typed as abc", "Hour left empty", "Hour 25", "Hour -1",
"Hour 9"};
  for (int i = 0; i < 5; ++i) {
    std::istringstream in(hourInputs[i]);
    int hour = 0;
    report(std::string("H0") + std::to_string(i + 1), hourNames[i], hourExpected[i],
        readHour(in, hour) ? "accepted" : "rejected");
  }

  // ---- Part D: five journeys, total worked out by hand ----
  // J1 Inner 07:30 off-peak 1000, J2 Middle 08:30 peak 2400, J3 Outer 12:00 off-peak 3000,
  // J4 Outer 18:00 peak 3600, J5 Inner 20:00 off-peak 1000  -> total 11000, balance 50000 -
11000 = 39000
  std::cout << "\nPart D: five sample journeys (starting balance Rs 500.00)\n";
  struct Trip { int zoneCode; int hour; int minute; };
  const Trip trips[] = {{0, 7, 30}, {1, 8, 30}, {2, 12, 0}, {2, 18, 0}, {0, 20, 0}};
  SmartCard card(50000);
  long long total = 0;
  for (const Trip& t : trips) {
    Zone z;
    zoneFromCode(t.zoneCode, z);
    long long f = fare(z, isPeak(t.hour, t.minute), Category::Regular);
    if (card.deduct(f)) total += f;
    std::cout << "  fare " << std::setw(5) << f << " paise, balance " << std::setw(6) <<
card.balance() << " paise\n";
  }
  report("J01", "Total of five journeys (paise)", "11000", std::to_string(total));
  report("J02", "Final balance (paise)", "39000", std::to_string(card.balance()));

  std::cout << "\nSummary: " << passed << " passed, " << failed << " failed, " << (passed +
failed) << " total\n";
  return failed == 0 ? 0 : 1;
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task09 Task09.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
9.4 Actual output
**Actual output (from execution)**

```
Part A: fare tests
F01  Inner off-peak 12:00                 expected 1000     actual 1000     PASS
F02  Inner peak 09:00                     expected 1200     actual 1200     PASS
F03  Middle off-peak 14:00                expected 2000     actual 2000     PASS
F04  Middle peak 18:30                    expected 2400     actual 2400     PASS
F05  Outer off-peak 22:00                 expected 3000     actual 3000     PASS
F06  Outer peak 17:30                     expected 3600     actual 3600     PASS
```

```
F07  Boundary 07:59 (off-peak)          expected 1000     actual 1000     PASS
F08  Boundary 08:00 (peak)              expected 1200     actual 1200     PASS
F09  Boundary 09:59 (peak)              expected 1200     actual 1200     PASS
F10  Boundary 10:00 (off-peak)          expected 1000     actual 1000     PASS
F11  Boundary 16:59 (off-peak)          expected 1000     actual 1000     PASS
F12  Boundary 17:00 (peak)              expected 1200     actual 1200     PASS
F13  Boundary 19:59 (peak)              expected 1200     actual 1200     PASS
F14  Boundary 20:00 (off-peak)          expected 1000     actual 1000     PASS
F15  Student Outer off-peak             expected 1500     actual 1500     PASS
F16  Student Outer peak                 expected 1800     actual 1800     PASS
F17  Senior Middle peak                 expected 1440     actual 1440     PASS
F18  Unknown zone code 5                expected -1       actual -1       PASS
F19  Unknown zone code -1               expected -1       actual -1       PASS

Part B: minimum balance tests
B01  Balance 6000, fare 1000 (leaves exactly 5000)  expected allowed   actual allowed   PASS
B02  Balance 5999, fare 1000 (leaves 4999)          expected denied    actual denied    PASS
B03  Balance 6000, fare 1001 (leaves 4999)          expected denied    actual denied    PASS
B04  Balance 10000, fare 3600 (leaves 6400)         expected allowed   actual allowed   PASS
B05  Balance 8000, fare 3600 (leaves 4400)          expected denied    actual denied    PASS

Part C: invalid input tests
R01  Recharge -500                          expected rejected  actual rejected  PASS
R02  Balance unchanged after bad recharge   expected 10000     actual 10000     PASS
R03  Recharge 0                             expected rejected  actual rejected  PASS
R04  Recharge 5000                          expected accepted  actual accepted  PASS
R05  Balance after valid recharge           expected 15000     actual 15000     PASS
H01  Hour typed as abc                      expected rejected  actual rejected  PASS
H02  Hour left empty                        expected rejected  actual rejected  PASS
H03  Hour 25                                expected rejected  actual rejected  PASS
H04  Hour -1                                expected rejected  actual rejected  PASS
H05  Hour 9                                 expected accepted  actual accepted  PASS

Part D: five sample journeys (starting balance Rs 500.00)
  fare 1000  paise, balance 49000  paise
  fare 2400  paise, balance 46600  paise
  fare 3000  paise, balance 43600  paise
  fare 3600  paise, balance 40000  paise
  fare 1000  paise, balance 39000  paise
J01  Total of five journeys (paise)     expected 11000     actual 11000     PASS
J02  Final balance (paise)              expected 39000     actual 39000     PASS

Summary: 36 passed, 0 failed, 36 total
```

[Screenshot to be inserted: Task09_Output.png - code and output from the compiler window]

9.5 Test table with expected and actual results

The tables below are the lines of the program output above, arranged as tables. The expected values were calculated by hand before they were written into the program (all values in paise; −1 means the journey is rejected).

Fare tests

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| F01 | Inner off-peak 12:00 | 1000 | 1000 | PASS |
| F02 | Inner peak 09:00 | 1200 | 1200 | PASS |
| F03 | Middle off-peak 14:00 | 2000 | 2000 | PASS |
| F04 | Middle peak 18:30 | 2400 | 2400 | PASS |
| F05 | Outer off-peak 22:00 | 3000 | 3000 | PASS |
| F06 | Outer peak 17:30 | 3600 | 3600 | PASS |
| F07 | Boundary 07:59 (off-peak) | 1000 | 1000 | PASS |
| F08 | Boundary 08:00 (peak) | 1200 | 1200 | PASS |
| F09 | Boundary 09:59 (peak) | 1200 | 1200 | PASS |
| F10 | Boundary 10:00 (off-peak) | 1000 | 1000 | PASS |
| F11 | Boundary 16:59 (off-peak) | 1000 | 1000 | PASS |
| F12 | Boundary 17:00 (peak) | 1200 | 1200 | PASS |
| F13 | Boundary 19:59 (peak) | 1200 | 1200 | PASS |
| F14 | Boundary 20:00 (off-peak) | 1000 | 1000 | PASS |
| F15 | Student Outer off-peak | 1500 | 1500 | PASS |
| F16 | Student Outer peak | 1800 | 1800 | PASS |
| F17 | Senior Middle peak | 1440 | 1440 | PASS |
| F18 | Unknown zone code 5 | -1 | -1 | PASS |
| F19 | Unknown zone code -1 | -1 | -1 | PASS |

Hand calculations used: Inner off-peak 1000; Inner peak 1000 + 200 = 1200; Middle peak 2000 + 400 = 2400; Outer peak 3000 + 600 = 3600. Student Outer off-peak 3000 × 50 / 100 = 1500; Student Outer peak 3600 × 50 / 100 = 1800; Senior Middle peak 2400 × 60 / 100 = 1440. The student and senior values depend on the assumed concession percentages.

Minimum balance tests

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| B01 | Balance 6000, fare 1000 (leaves exactly 5000) | allowed | allowed | PASS |
| B02 | Balance 5999, fare 1000 (leaves 4999) | denied | denied | PASS |
| B03 | Balance 6000, fare 1001 (leaves 4999) | denied | denied | PASS |
| B04 | Balance 10000, fare 3600 (leaves 6400) | allowed | allowed | PASS |
| B05 | Balance 8000, fare 3600 (leaves 4400) | denied | denied | PASS |

B01 is the exact limit: 6000 − 1000 = 5000, which is allowed because the rule is "at least" 5000. B02 and B03 are one paisa below the limit and are refused.

Invalid input tests

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| R01 | Recharge -500 | rejected | rejected | PASS |

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| R02 | Balance unchanged after bad recharge | 10000 | 10000 | PASS |
| R03 | Recharge 0 | rejected | rejected | PASS |
| R04 | Recharge 5000 | accepted | accepted | PASS |
| R05 | Balance after valid recharge | 15000 | 15000 | PASS |
| H01 | Hour typed as abc | rejected | rejected | PASS |
| H02 | Hour left empty | rejected | rejected | PASS |
| H03 | Hour 25 | rejected | rejected | PASS |
| H04 | Hour -1 | rejected | rejected | PASS |
| H05 | Hour 9 | accepted | accepted | PASS |

Five sample journeys (starting balance Rs 500.00 = 50000 paise)

| No. | Journey | Peak? | Fare by hand (paise) | Balance after (paise) |
|---|---|---|---|---|
| 1 | Inner, 07:30 | No | 1000 | 49000 |
| 2 | Middle, 08:30 | Yes | 2000 + 400 = 2400 | 46600 |
| 3 | Outer, 12:00 | No | 3000 | 43600 |
| 4 | Outer, 18:00 | Yes | 3000 + 600 = 3600 | 40000 |
| 5 | Inner, 20:00 | No (20:00 is off-peak) | 1000 | 39000 |
| Total | | | 1000 + 2400 + 3000 + 3600 + 1000 = 11000 | 50000 − 11000 = 39000 |

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| J01 | Total of five journeys (paise) | 11000 | 11000 | PASS |
| J02 | Final balance (paise) | 39000 | 39000 | PASS |

The per-journey fares and balances printed in Part D of the output (1000, 2400, 3000, 3600, 1000 and 49000, 46600, 43600, 40000, 39000) match the hand calculation exactly.
**Result:** 36 tests were run and all 36 passed (see the Summary line of the output). One limitation was noticed while writing the tests: cin >> hour with input such as 9.5 reads 9 and leaves .5, as shown for 7x in Task 8. This is not covered by the table and would need the stricter getline() + stoi() reading.
9.6 Alternative concept: table of test cases in a loop
The program above already uses a table. The alternative would be hand-written checks, one after another, for example:

```
if (fare(Zone::Inner, isPeak(8, 0), Category::Regular) != 1200) std::cout << "F08 FAIL\n";
if (fare(Zone::Inner, isPeak(10, 0), Category::Regular) != 1000) std::cout << "F10 FAIL\n";
// ... one such statement for every case
```

| Point | Table of cases in a loop (used) | Separate hand-written checks |
|---|---|---|
| Time for t tests | O(t): one loop, each test O(1) | O(t): t statements, each O(1) |

| Point | Table of cases in a loop (used) | Separate hand-written checks |
|---|---|---|
| Space | The table: t entries, each a small structure (zone, hour, minute, category, expected). This is O(t) data | No table, but O(t) lines of code |
| Adding a new boundary case | One new row | One new statement, copied and edited by hand (easy to make a typing mistake) |
| Reporting | Same PASS/FAIL format for every case | Depends on how each check was written |
| Flexibility | The table could be read from a file without recompiling | Needs recompiling |

9.7 Conclusion

The fare logic gave the expected result in all 36 test cases, including the four time boundaries, the exact Rs 50 limit, and the invalid inputs. The table-driven method makes it easy to add more boundary cases.

Task 10 – Compare Expectations with Execution (4 Marks)

10.1 Requirement

Compare what was expected with what happened: whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree and by how much they differ, whether int overflows for the network total, what warnings -Wall -Wextra gave, and the reasons for any difference. The alternative concept is accumulating in int, long long or double.

10.2 Assumptions and method

Journey number i uses zone i % 3 (Inner, Middle, Outer), peak when i % 6 is 3 or more, and passenger category i % 3 (regular 0%, student 50%, senior 40%). The mix is an assumption made to get realistic fares with different values; it is not data from the assignment.

One block of six journeys costs 1000 + 1000 + 1800 + 1200 + 1200 + 2160 = 8360 paise (calculated by hand), so 3,000,000 journeys (500,000 blocks) are expected to cost 500,000 × 8360 = 4,180,000,000 paise. The program must give this number.

The integer total is the exact reference. The double total adds fares in rupees, with the 20% surcharge written as * 1.2 and the concession as * (1.0 - pct / 100.0), which is how a careless first version could be written.

To show what a 32-bit int would hold, the sum is also kept in an unsigned int (wrap-around is defined for unsigned types). Real signed overflow is undefined behaviour in C++, so it was not executed.

10.3 Source code

**C++ source code (file: Task10/Task10.cpp)**

```
// Task 10: integer paise against double rupees, and int / long long / double accumulators
#include <chrono>
#include <climits>
#include <iomanip>
#include <iostream>

// journey i: type i % 6 (Inner/Middle/Outer off-peak, then Inner/Middle/Outer peak),
// category i % 3 (Regular 0%, Student 50%, Senior 40% concession)
constexpr int BASE_RUPEES[3] = {10, 20, 30};
constexpr int CONCESSION[3] = {0, 50, 40};
```

```
long long farePaise(long long i)
{
   int zone = static_cast<int>(i % 3);
   bool peak = (i % 6) >= 3;
   long long f = BASE_RUPEES[zone] * 100LL;
   if (peak) f += f * 20 / 100;
   return f * (100 - CONCESSION[i % 3]) / 100;
}

double fareRupeesDouble(long long i)
{
   int zone = static_cast<int>(i % 3);
   bool peak = (i % 6) >= 3;
   double f = BASE_RUPEES[zone];
   if (peak) f *= 1.2;                    // 20% surcharge written with a double
   return f * (1.0 - CONCESSION[i % 3] / 100.0);
}

int main()
{
   const long long counts[] = {20000, 500000, 3000000};
   std::cout << std::fixed;
   for (long long n : counts) {
      long long paiseTotal = 0;
      double rupeeTotal = 0.0;
      double paiseInDouble = 0.0;
      unsigned int intLike = 0;       // simulates a 32-bit int without undefined behaviour
      for (long long i = 0; i < n; ++i) {
         long long f = farePaise(i);
         paiseTotal += f;
         rupeeTotal += fareRupeesDouble(i);
         paiseInDouble += static_cast<double>(f);
         intLike += static_cast<unsigned int>(f);
      }
      std::cout << "n = " << n << '\n';
      std::cout << "  long long paise total      : " << paiseTotal << " paise (Rs "
            << paiseTotal / 100 << '.' << std::setw(2) << std::setfill('0') << paiseTotal % 100
            << std::setfill(' ') << ")\n";
      std::cout << "  double rupee total         : " << std::setprecision(10) << rupeeTotal << "
Rs\n";
      std::cout << "  double rupee total x 100   : " << std::setprecision(10) << rupeeTotal * 100.0
<< " paise\n";
      std::cout << "  difference (double - exact): " << std::setprecision(10)
            << rupeeTotal * 100.0 - static_cast<double>(paiseTotal) << " paise\n";
      std::cout << "  double accumulating paise  : " << std::setprecision(1) << paiseInDouble
<< " paise\n";
      std::cout << "  exceeds INT_MAX            : " << std::boolalpha << (paiseTotal >
INT_MAX) << '\n';
      std::cout << "  32-bit int would hold      : " << static_cast<int>(intLike) << " paise\n";
   }
```

```
    // time per addition (measured with std::chrono on this machine)
    const long long N = 3000000;
    const int REPEAT = 20;
    {
        auto t0 = std::chrono::steady_clock::now();
        for (int r = 0; r < REPEAT; ++r) { unsigned int s = 0; for (long long i = 0; i < N; ++i) s
+= 1200u; volatile unsigned int sink = s; (void)sink; }
        auto t1 = std::chrono::steady_clock::now();
        std::cout << "\nint-sized addition      : " << std::setprecision(3)
                  <<       std::chrono::duration<double,       std::nano>(t1       -       t0).count()       /
(static_cast<double>(N) * REPEAT) << " ns per addition\n";
    }
    {
        auto t0 = std::chrono::steady_clock::now();
        for (int r = 0; r < REPEAT; ++r) { long long s = 0; for (long long i = 0; i < N; ++i) s +=
1200; volatile long long sink = s; (void)sink; }
        auto t1 = std::chrono::steady_clock::now();
        std::cout << "long long addition      : "
                  <<       std::chrono::duration<double,       std::nano>(t1       -       t0).count()       /
(static_cast<double>(N) * REPEAT) << " ns per addition\n";
    }
    {
        auto t0 = std::chrono::steady_clock::now();
        for (int r = 0; r < REPEAT; ++r) { double s = 0; for (long long i = 0; i < N; ++i) s += 12.0;
volatile double sink = s; (void)sink; }
        auto t1 = std::chrono::steady_clock::now();
        std::cout << "double addition         : "
                  <<       std::chrono::duration<double,       std::nano>(t1       -       t0).count()       /
(static_cast<double>(N) * REPEAT) << " ns per addition\n";
    }
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task10 Task10.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
10.4 Actual output

**Actual output (from execution)**

```
n = 20000
  long long paise total      : 27865880 paise (Rs 278658.80)
  double rupee total         : 278658.8000000083 Rs
  double rupee total x 100   : 27865880.0000008270 paise
  difference (double - exact): 0.0000008270 paise
  double accumulating paise  : 27865880.0 paise
  exceeds INT_MAX            : false
  32-bit int would hold      : 27865880 paise
n = 500000
  long long paise total      : 696665880 paise (Rs 6966658.80)
  double rupee total         : 6966658.7999909446 Rs
  double rupee total x 100   : 696665879.9990944862 paise
  difference (double - exact): -0.0009055138 paise
  double accumulating paise  : 696665880.0 paise
  exceeds INT_MAX            : false
```

```
  32-bit int would hold      : 696665880 paise
n = 3000000
  long long paise total      : 4180000000 paise (Rs 41800000.00)
  double rupee total         : 41800000.0003932416 Rs
  double rupee total x 100   : 4180000000.0393242836 paise
  difference (double - exact): 0.0393242836 paise
  double accumulating paise  : 4180000000.0 paise
  exceeds INT_MAX            : true
  32-bit int would hold      : -114967296 paise

int-sized addition      : 1.012 ns per addition
long long addition      : 0.765 ns per addition
double addition         : 2.873 ns per addition
```

[Screenshot to be inserted: Task10_Output.png - code and output from the compiler window]

10.5 Expected against observed

| Journeys | Exact total (long long paise) | double rupees × 100 (paise) | Difference (paise) | Exceeds INT_MAX? | What a 32-bit int would hold |
|---|---|---|---|---|---|
| 20,000 | 27,865,880 | 27,865,880.000000827 | +0.00000083 | No | 27,865,880 |
| 500,000 | 696,665,880 | 696,665,879.999094486 | −0.00091 | No | 696,665,880 |
| 3,000,000 | 4,180,000,000 | 4,180,000,000.039324284 | +0.039 | Yes | −114,967,296 |

The 3,000,000-journey total matches the hand-calculated 4,180,000,000 paise (Rs 4,18,00,000.00).

10.6 Discussion of the differences

**Do the double and the integer totals agree?** Yes to within a very small amount: the difference is below 0.04 paise even at 3,000,000 journeys. Rounded to the nearest paisa the two agree. The difference is not zero and it is not the same sign: +0.00000083, −0.00091 and +0.039 paise, so it is random-looking and grows with the number of journeys, as seen in Task 3.

**Why do they differ?** Fares such as 1.2 × 10 or 0.6 × 36 are not exactly representable in binary floating point, and each addition also rounds. The integer version has no fractions at all, so it has no rounding.

**Why a tiny difference still matters.** For 500,000 journeys the double result is 696,665,879.999094 paise. If a program converts it to whole paise with static_cast<long long> the fraction is truncated and the result is 696,665,879, which is one paisa less than the true total 696,665,880. Comparing the two totals with == would also say they are different. These are the "off by a paisa" faults in the first version. (This sentence is calculated from the printed value.)

**Does the double lose accuracy when it holds whole paise?** No. The line "double accumulating paise" in the output equals the exact total (27,865,880.0, 696,665,880.0 and 4,180,000,000.0), because integers up to about 9.0×10^15 are exact in a double. The problem is the fractions, not the type itself.

**Does int overflow?** For 20,000 and 500,000 journeys it does not (the totals are 27.9 million and 696.7 million, below 2,147,483,647). For the whole network it does: 4,180,000,000 is larger than INT_MAX. A 32-bit int would wrap to −114,967,296, a negative collection, with no warning at run time. With the mixed fares of this assumption (average 4,180,000,000 ÷ 3,000,000 ≈ 1393 paise), int would overflow after about 1.54 million journeys (2,147,483,647

÷ 1393.33), which is about half of the network day and three times the one-line figure of 500,000. So int is enough for one station and for one line, and not enough for the network.

10.7 Compiler warnings with -Wall -Wextra

The compiler gave no warning for Task10.cpp. The warnings actually produced by the programs of this assignment are listed below. Each is copied from the compiler output saved with the program.

| Program | Warning | Meaning |
|---|---|---|
| Task04.cpp | comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare] | The int is converted to unsigned, so −1 becomes a huge number. The comparison gives a surprising result (Task 4) |
| Task05_incdemo.cpp | operation on 'i' may be undefined [-Wsequence-point] (printed twice) | i = i++ changes i twice in one expression |
| Task12_COMPILEONLY_nodefault.cpp | enumeration value 'Airport' not handled in switch [-Wswitch] and control reaches end of non-void function [-Wreturn-type] | A new enumerator was added but the switch (with no default) does not handle it, and the function can run off its end (Task 12) |

No warning was produced for the int overflow of the network total, because this happens only at run time with real data. This is the reason why the type must be chosen correctly in advance and cannot be left to the compiler.

10.8 Alternative concept: int, long long or double accumulator

Time per addition was measured with std::chrono::steady_clock over 3,000,000 additions repeated 20 times, compiled without optimisation, on a single-core Linux system (the same system as all other results). The int case used unsigned int (see 10.2). These are small numbers on one machine and will vary from run to run; they show the order of magnitude only.

| Accumulator | Behaviour at 3,000,000 journeys | Fails when | Time per addition (measured) | Space |
|---|---|---|---|---|
| int (32-bit) | Overflows: −114,967,296 paise | Total above 2,147,483,647 paise (about 1.54 million journeys of this mix) | 1.012 ns | 4 bytes |
| long long | Exact: 4,180,000,000 paise | Total above 9.22×10^18 paise (about 6.6×10^15 journeys of this mix) | 0.765 ns | 8 bytes |
| double (rupees with fractions) | No overflow, but 0.039 paise error | Never overflows in practice. It loses precision as fractions and size grow | 2.873 ns | 8 bytes |

All three are O(1) per addition. The measured differences are about one to three nanoseconds, and the order of int and long long is not stable: in an earlier run of the same program the values were 1.03 ns (int), 1.20 ns (long long) and 3.35 ns (double), so differences of this size between int and long long are measurement noise. The floating-point addition was the slowest in both runs, by a factor of about three, probably because of its longer latency, but I did not investigate the reason. Adding 3,000,000 values to a double costs about 9 ms at -O0, which is not important for the gate. long long costs 4 more bytes than int and removes the overflow problem completely.

10.9 Conclusion

The expectation that integer paise give the exact total (4,180,000,000) was confirmed, the double result was close but not exact (+0.039 paise), and int overflowed for the whole network. These measurements support the choice of long long paise.

Task 11 – Recommend and Defend the Final Design (4 Marks)

11.1 Requirement

Recommend the final design (types, constants, classes), explain how a fare revision is carried out, state the limitations, and say whether the design stays correct for the whole network, based on evidence from my own experiments. The alternative concept is reading the fare table from a configuration file at start-up.

11.2 Recommended design

| **Item** | **Recommendation** | **Evidence from this report** |
|---|---|---|
| Money | long long in paise for fares, balances and totals. double only for printing | Task 3: 0.29 × 100 lost a paisa; Task 10: the double total was off by 0.039 paise and a truncated total was one paisa short |
| Totals | long long | Task 3 and 10: int overflowed for the network (4,180,000,000 > 2,147,483,647) |
| Constants | constexpr for fares, surcharge, concessions and minimum balance, all in one place | Task 2: type checking, no run-time cost |
| Zones | enum class Zone with a fare table indexed by the zone | Task 2 and 6: no implicit conversion; Task 12: the fourth zone was added with one table entry |
| Time rule | Minutes from midnight with half-open intervals [8:00, 10:00) and [17:00, 20:00) | Task 6 and 9: 8:00 and 17:00 peak, 10:00 and 20:00 off-peak, all boundary tests passed |
| Arithmetic | Multiply first, then divide (f * pct / 100) | Task 5 and 9: exact fares, 36 of 36 tests passed |
| Classes | SmartCard (private balance, canTravel, deduct, recharge), Passenger with derived classes, Journey, FareTable | Task 1 and 7: invalid balance changes are impossible through the class |
| Input | Read lines with getline() and stoi(), check the whole line and the range | Task 8: 7x was caught only by this method |
| Testing | A table of test cases in a loop, run after every change | Task 9 and 12 |

11.3 How a fare revision is carried out

Change the value in the one place where fares are defined (the constants block, or the configuration file described in 11.6).

Rebuild (if constants are compiled in), then run the test table. Tests whose expected values depend on the old fares are updated by hand first, so that the tests check the new rule and not the old one.

Check the boundary rows and the exact Rs 50 row again.

Deploy the new version to the gates.

11.4 Limitations

The concession percentages (50% for students, 40% for senior citizens) and the order of surcharge and concession are my assumptions, because the assignment does not state them. They must be confirmed with the metro authority.

Integer division drops fractions of a paisa. With the present fares and percentages every result is a whole number of paise (all fares are multiples of 10 paise), but a future percentage such as 15% of 1000 paise is fine while 7% of 1005 paise is not. The rounding rule would have to be decided.

The program handles one gate and one card at a time. With many gates sharing one balance, two simultaneous journeys would need a transaction or a lock on the card record. This was not designed or tested.

There is no limit on the recharge amount and no check that a recharge would overflow long long (practically impossible, but not checked).

Time is given as an hour and minute; time zones, public holidays and clock errors at the gate are not modelled.

All timing numbers in this report were measured on one single-core system without optimisation, so they only show the order of magnitude.

11.5 Is the design correct for the whole network?

Yes for correctness of the money. The fare per journey is O(1), so 3,000,000 journeys a day is about 35 journeys per second on average (3,000,000 ÷ 86,400), which is a very small load for the calculation itself. The only value that outgrew 32 bits was the network total, and it is stored as long long. The real limits of the network will be in the card database and gate communication, which were not part of this assignment.

11.6 Alternative concept: fare table read from a configuration file

The program below reads the fares, the surcharge percentage and the minimum balance from a text file when it starts and validates every value. A missing key, a negative fare or an unknown key stops the start-up.

**Configuration file fares.cfg**

```
INNER 1000
MIDDLE 2000
OUTER 3000
PEAK_SURCHARGE_PERCENT 20
MIN_BALANCE_PAISE 5000
```

**Invalid configuration file fares_bad.cfg (negative Middle fare)**

```
INNER 1000
MIDDLE -2000
OUTER 3000
PEAK_SURCHARGE_PERCENT 20
MIN_BALANCE_PAISE 5000
```

**C++ source code (file: Task11/Task11.cpp)**

```
// Task 11: fare table read from a configuration file when the gate starts
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>

struct FareConfig {
   long long base[3] = {0, 0, 0};
   int surchargePercent = 0;
   long long minBalance = 0;
};

// returns false if a key is missing, unknown or has an invalid value
bool loadConfig(const std::string& path, FareConfig& cfg)
{
   std::ifstream in(path);
   if (!in) return false;
   bool seen[5] = {false, false, false, false, false};
   std::string key;
   long long value;
   while (in >> key >> value) {
      if (key == "INNER" && value > 0) { cfg.base[0] = value; seen[0] = true; }
      else if (key == "MIDDLE" && value > 0) { cfg.base[1] = value; seen[1] = true; }
      else if (key == "OUTER" && value > 0) { cfg.base[2] = value; seen[2] = true; }
      else if (key == "PEAK_SURCHARGE_PERCENT" && value >= 0 && value <= 100) {
cfg.surchargePercent = static_cast<int>(value); seen[3] = true; }
      else if (key == "MIN_BALANCE_PAISE" && value >= 0) { cfg.minBalance = value;
seen[4] = true; }
      else return false;
   }
   for (bool s : seen) if (!s) return false;
   return true;
}

int main(int argc, char* argv[])
{
   std::string path = (argc > 1) ? argv[1] : "fares.cfg";
   FareConfig cfg;
   auto t0 = std::chrono::steady_clock::now();
   bool ok = loadConfig(path, cfg);
   auto t1 = std::chrono::steady_clock::now();
   if (!ok) {
      std::cout << "Configuration file '" << path << "' is missing or invalid. Gate not started.\n";
      return 1;
   }
   std::cout << "Loaded '" << path << "'\n";
   const char* names[] = {"Inner", "Middle", "Outer"};
   for (int i = 0; i < 3; ++i) {
      long long peakFare = cfg.base[i] + cfg.base[i] * cfg.surchargePercent / 100;
```

```
        std::cout << names[i] << ": off-peak " << cfg.base[i] << " paise, peak " << peakFare << "
paise\n";
    }
    std::cout << "Minimum balance: " << cfg.minBalance << " paise\n";

    // average load time over many repeats
    const int REPEAT = 2000;
    auto s0 = std::chrono::steady_clock::now();
    for (int r = 0; r < REPEAT; ++r) { FareConfig tmp; loadConfig(path, tmp); }
    auto s1 = std::chrono::steady_clock::now();
    std::cout << "First load took " << std::chrono::duration<double, std::micro>(t1 - t0).count()
<< " microseconds\n";
    std::cout << "Average over " << REPEAT << " loads: "
              << std::chrono::duration<double, std::micro>(s1 - s0).count() / REPEAT << "
microseconds\n";
    std::cout << "sizeof(FareConfig) = " << sizeof(FareConfig) << " bytes\n";
}
```

Compile and run: g++ -std=c++17 -Wall -Wextra -o Task11 Task11.cpp, then ./Task11 fares.cfg and ./Task11 fares_bad.cfg.
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output with fares.cfg**

```
Loaded 'fares.cfg'
Inner: off-peak 1000 paise, peak 1200 paise
Middle: off-peak 2000 paise, peak 2400 paise
Outer: off-peak 3000 paise, peak 3600 paise
Minimum balance: 5000 paise
First load took 50.896 microseconds
Average over 2000 loads: 4.09606 microseconds
sizeof(FareConfig) = 40 bytes
```

**Actual output with fares_bad.cfg**

```
Configuration file 'fares_bad.cfg' is missing or invalid. Gate not started.
```

[Screenshot to be inserted: Task11_Output.png - code and output from the compiler window]

**Time taken to compile Task06.cpp five times (measured with date before and after g++)**

```
compile 1: .347568103 s
compile 2: .411080406 s
compile 3: .324227397 s
compile 4: .337236797 s
compile 5: .321215210 s
```

| Point | Constants compiled into the program | Fare table read from a file |
|---|---|---|
| Start-up time | None | O(z) to read z fare values. Measured: first load 50.896 microseconds, average over 2000 loads 4.09606 microseconds (5 values; the first read is slower because the file is not yet cached; an earlier run gave 51 and 3.2 microseconds) |
| Space | The constants are in the program (24 bytes for 3 fares) | A struct of 40 bytes (sizeof(FareConfig) = 40) for 3 fares, the percentage and the minimum balance, O(z) |
| Fare revision | Edit, recompile (about 0.32–0.41 s of compiler time for this small program), test, redeploy the new program to every gate | Edit one line of the text file and restart or reload. No recompiling. The redeployment of a small file is still needed |
| Safety | The compiler checks types and static_assert can check the table | A wrong file gives wrong fares unless the program validates it (as shown with the invalid file). The program refused to start with the bad file |
| Auditing | Fares are part of the source code under version control | The file must be put under version control and signed or checked |

The extra start-up cost is at most a few hundred microseconds (a few microseconds once the file is cached) and 40 bytes, which is negligible against the cost of recompiling and redeploying the whole program (and its approval and testing) for every fare change. The measured compile time on its own (about 0.32 to 0.41 seconds) is also small, but the compile time is only a small part of the real cost: the compiled program has to be tested again and installed on every gate.

11.7 Final recommendation

Keep the rule logic (peak intervals, surcharge formula, minimum balance rule, class structure) compiled in, because it changes rarely and the compiler can check it. Put the numbers that change with a fare revision (base fares, surcharge percent, concession percent, minimum balance) in a validated configuration file, with the same values as compiled-in defaults, and run the test table after every change. The design uses long long paise, constexpr defaults, enum class zones and a SmartCard with a private balance.

Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge, 8 Marks)

12.1 Changed requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones. The original design may be retained, modified or redesigned.

**Assumptions made because the changed requirement is not complete.** (1) On Saturday and Sunday the fare is the base fare minus 10%, and the peak surcharge is not applied even in peak hours. (2) On weekdays the rules are unchanged. (3) A concession (student 50%, senior 40%) is applied after the weekend discount or the surcharge. (4) A pass belongs to one calendar month; a journey is covered when its farthest zone is one of the two chosen zones. (5) A covered journey costs nothing and does not need the Rs 50 minimum balance (the card is not charged). (6) A journey outside the pass zones, or in another month, is charged normally, with all the old rules. These assumptions can be changed by editing one function each. [Confirm these assumptions with the faculty if needed]

A. Impact Analysis (2 Marks)

| Part of the design | Affected? | What has to change |
|---|---|---|
| Constants | Yes | BASE_PAISE gets a fourth value, 6000. ZONE_COUNT goes from 3 to 4 (better: it is taken from the enum). New constants are needed: WEEKEND_DISCOUNT_PERCENT = 10. The surcharge, concessions and minimum balance stay |
| Enum values | Yes | Zone gets Airport (and a Count value to give the size). New enums are needed for the day of the week (weekend or not) |
| Expressions | Yes | The fare expression had two cases (peak, off-peak). It now has three: weekend (discount, no surcharge), weekday peak (surcharge) and weekday off-peak. The surcharge expression f += f * 20 / 100 must be skipped on weekends |
| Switch statements | Yes | switch (z) needs a case Zone::Airport. A forgotten case is silently an error value if there is a default (shown below) |
| The peak check | No | isPeak(hour, minute) is unchanged and still decides the weekday surcharge |
| SmartCard | Partly | The private balance, canTravel(), deduct() and the minimum balance rule stay. The card now must hold an optional pass, and a pass journey must skip the charge |
| Journey | Yes | The journey must carry the day of the week and the date (month and year) so the discount and the pass can be decided |
| New class | Yes | A Pass class: two chosen zones, month and year, and covers(journey) |
| Tests | Yes | New tests for Airport, weekend, pass and expired pass; the old tests must still pass |

B. Evaluation of the Original Design (2 Marks)

To check whether the original design can absorb the changes, the original fare() and canTravel() from Task 6 were taken, only the enumerator Airport was added to Zone, and a journey in the Airport zone was tried with Rs 200 on the card.

**C++ source code (file: Task12/Task12_impact.cpp)**

```
// Task 12 A/B: the ORIGINAL switch-based design with only the new enum value added
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer, Airport };   // Airport added to the enum only

constexpr int BASE_PAISE[] = {1000, 2000, 3000};      // still three entries
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
   long long base = 0;
   switch (z) {
      case Zone::Inner:  base = BASE_PAISE[0]; break;
```

```
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 20000;   // Rs 200.00, enough for an Airport fare
    std::cout << "Airport off-peak fare returned: " << fare(Zone::Airport, false) << '\n';
    std::cout << "Airport journey allowed: " << std::boolalpha << canTravel(balance,
fare(Zone::Airport, false)) << '\n';
    std::cout << "Weekend discount: no place to put it in fare(Zone, bool)\n";
    std::cout << "Monthly pass: SmartCard has no field for a pass\n";
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task12_impact Task12_impact.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).

**Actual output (from execution)**

```
Airport off-peak fare returned: -1
Airport journey allowed: false
Weekend discount: no place to put it in fare(Zone, bool)
Monthly pass: SmartCard has no field for a pass
```

**Worked example 1 – fourth zone.** The Airport journey returned the fare −1 and was refused although the card had Rs 200. The compiler gave no warning, because the default: branch of the switch handles every other value. The default that was added for safety hides the missing case. So the original code cannot absorb the new zone without an edit of the switch, and it fails quietly if the edit is forgotten. A second file (Task12_COMPILEONLY_nodefault.cpp), the same switch without a default, was compiled only:

**Compiler messages with -Wall -Wextra**

```
Task12_COMPILEONLY_nodefault.cpp: In function 'long long int baseFare(Zone)':
Task12_COMPILEONLY_nodefault.cpp:9:12: warning: enumeration value 'Airport' not
handled in switch [-Wswitch]
    9 |     switch (z) {
      |            ^
Task12_COMPILEONLY_nodefault.cpp:14:1: warning: control reaches end of non-void
function [-Wreturn-type]
   14 | }
      | ^
```

Here the compiler does warn (-Wswitch), but the function then has no value to return for Airport (-Wreturn-type), so this version is also wrong. Either way the switch needs an edit, and the safe default hides the omission.

**Worked example 2 – weekend discount.** An Outer journey on Sunday at 18:00 should cost 3000 − 10% = 2700 paise. The original fare(Zone, bool peak) has no day. If the caller passes peak = true it returns 3600; if the caller passes peak = false it returns 3000. Neither is 2700, and the function cannot know that the discount applies. The rule "discount instead of surcharge" cannot be written without a new parameter or a separate function, so the fare() signature must change.

**Worked example 3 – monthly pass.** SmartCard holds only a balance. A pass cannot be stored in it. If a pass journey is given the fare 0, canTravel(3000, 0) is 3000 − 0 >= 5000, which is false, so a pass holder with Rs 30 on the card would be refused although the journey is free. This needs a design decision (assumption 5) and a change in the card.

**Evaluation.** What can be kept: integer paise, long long, constexpr constants, enum class zones, the isPeak() function, and the SmartCard class with its private balance and the canTravel() / deduct() rules (these still enforce R1 and R2). What cannot be kept: the switch-based fare function (needs the day and a place for the discount), and the idea that a card has only a balance. The original design is therefore **neither fully sufficient nor in need of a complete rewrite**.

C. Adaptation or Redesign (2 Marks)

**Decision: MODIFY.** The fare table is now an array indexed by the zone (its size comes from Zone::Count), the discount rule is a separate step in calculateFare(), the journey information is collected in a JourneyInfo structure, a new Pass class is added, and SmartCard keeps an optional Pass (std::optional, C++17). The fare for a journey is calculated as:

base = BASE_PAISE[zone]
if weekend: fare = base - base * 10 / 100 (no surcharge)
else if peak hour: fare = base + base * 20 / 100
else: fare = base
then the concession: fare = fare * (100 - concession) / 100
pass covers journey -> charge 0 (no balance check); otherwise canTravel(fare) and deduct(fare)

**Revised C++ source code (file: Task12/Task12.cpp)**

```
// Task 12: revised design - fare table indexed by zone, separate discount rule, Pass class
#include <iomanip>
#include <iostream>
#include <optional>
#include <string>

enum class Zone { Inner, Middle, Outer, Airport, Count };   // Count = number of zones
enum class Category { Regular, Student, Senior };
enum class Day { Mon, Tue, Wed, Thu, Fri, Sat, Sun };

constexpr int ZONE_COUNT = static_cast<int>(Zone::Count);
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000, 2000, 3000, 6000};   // Rs 10, 20, 30, 60
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr int STUDENT_CONCESSION_PERCENT = 50;   // assumed
constexpr int SENIOR_CONCESSION_PERCENT = 40;    // assumed
constexpr long long MIN_BALANCE_PAISE = 5000;

struct JourneyInfo {
```

```
    Zone zone;      // farthest zone travelled
    int hour;
    int minute;
    Day day;
    int year;
    int month;
};

bool isPeak(int hour, int minute)
{
    int m = hour * 60 + minute;
    return (m >= 8 * 60 && m < 10 * 60) || (m >= 17 * 60 && m < 20 * 60);
}

bool isWeekend(Day d) { return d == Day::Sat || d == Day::Sun; }

// Discount rule kept apart from the fare table.
// Weekend: 10% off, and the peak surcharge is NOT applied. Weekday: surcharge in peak hours.
long long calculateFare(const JourneyInfo& j, Category c)
{
    long long f = BASE_PAISE[static_cast<int>(j.zone)];
    if (isWeekend(j.day)) {
        f -= f * WEEKEND_DISCOUNT_PERCENT / 100;
    } else if (isPeak(j.hour, j.minute)) {
        f += f * PEAK_SURCHARGE_PERCENT / 100;
    }
    if (c == Category::Student) f = f * (100 - STUDENT_CONCESSION_PERCENT) / 100;
    if (c == Category::Senior) f = f * (100 - SENIOR_CONCESSION_PERCENT) / 100;
    return f;
}

// Monthly pass: unlimited journeys when the farthest zone is one of the two chosen zones
class Pass {
public:
    Pass(Zone a, Zone b, int y, int m) : zoneA(a), zoneB(b), year(y), month(m) {}
    bool covers(const JourneyInfo& j) const
    {
        return j.year == year && j.month == month && (j.zone == zoneA || j.zone == zoneB);
    }
private:
    Zone zoneA, zoneB;
    int year, month;
};

enum class Outcome { Paid, PassUsed, Denied };

class SmartCard {
public:
    explicit SmartCard(long long paise) : balancePaise(paise) {}
    void addPass(const Pass& p) { pass = p; }
```

```
    long long balance() const { return balancePaise; }

    bool canTravel(long long f) const { return f >= 0 && balancePaise - f >= MIN_BALANCE_PAISE; }

    Outcome travel(const JourneyInfo& j, Category c, long long& charged)
    {
        charged = 0;
        if (pass && pass->covers(j)) return Outcome::PassUsed;    // assumption: no charge, no balance check
        long long f = calculateFare(j, c);
        if (!canTravel(f)) return Outcome::Denied;
        balancePaise -= f;
        charged = f;
        return Outcome::Paid;
    }
private:
    long long balancePaise;
    std::optional<Pass> pass;
};

int passed = 0, failed = 0;

void report(const std::string& id, const std::string& name, const std::string& expected,
            const std::string& actual)
{
    bool ok = (expected == actual);
    if (ok) ++passed; else ++failed;
    std::cout << std::left << std::setw(5) << id << std::setw(46) << name
              << "expected " << std::setw(10) << expected << "actual " << std::setw(10) << actual
              << (ok ? "PASS" : "FAIL") << '\n';
}

std::string outcomeText(Outcome o, long long charged)
{
    if (o == Outcome::PassUsed) return "pass,0";
    if (o == Outcome::Denied) return "denied";
    return "paid," + std::to_string(charged);
}

int main()
{
    // ---- fare tests (expected values worked out by hand, in paise) ----
    struct FareCase { const char* id; const char* name; JourneyInfo j; Category cat; long long expected; };
    const FareCase fareCases[] = {
        {"A01", "Airport weekday off-peak (Mon 12:00)", {Zone::Airport, 12, 0, Day::Mon, 2026, 10}, Category::Regular, 6000},
        {"A02", "Airport weekday peak (Mon 09:00)", {Zone::Airport, 9, 0, Day::Mon, 2026, 10}, Category::Regular, 7200},
```

```
        {"A03", "Airport weekend off-peak (Sat 12:00)", {Zone::Airport, 12, 0, Day::Sat, 2026,
10}, Category::Regular, 5400},
        {"A04", "Airport weekend peak hour (Sat 09:00)", {Zone::Airport, 9, 0, Day::Sat, 2026,
10}, Category::Regular, 5400},
        {"A05", "Outer weekend peak hour (Sun 18:00)", {Zone::Outer, 18, 0, Day::Sun, 2026,
10}, Category::Regular, 2700},
        {"A06", "Inner weekend off-peak (Sun 12:00)", {Zone::Inner, 12, 0, Day::Sun, 2026, 10},
Category::Regular, 900},
        {"A07", "Middle weekday peak 17:00 (old rule)", {Zone::Middle, 17, 0, Day::Fri, 2026,
10}, Category::Regular, 2400},
        {"A08", "Outer weekday 10:00 off-peak (old rule)", {Zone::Outer, 10, 0, Day::Wed,
2026, 10}, Category::Regular, 3000},
        {"A09", "Outer weekday 20:00 off-peak (old rule)", {Zone::Outer, 20, 0, Day::Wed,
2026, 10}, Category::Regular, 3000},
        {"A10", "Student Inner weekend off-peak", {Zone::Inner, 12, 0, Day::Sun, 2026, 10},
Category::Student, 450},
        {"A11", "Senior Airport weekday peak", {Zone::Airport, 9, 0, Day::Tue, 2026, 10},
Category::Senior, 4320},
    };
    std::cout << "Part A: fare calculation tests\n";
    for (const FareCase& t : fareCases)
        report(t.id, t.name, std::to_string(t.expected), std::to_string(calculateFare(t.j, t.cat)));

    // ---- card and pass tests ----
    struct CardCase { const char* id; const char* name; long long balance; bool hasPass; Zone
pa; Zone pb;
                      int passMonth; JourneyInfo j; const char* expected; };
    const CardCase cardCases[] = {
        {"C01", "Pass Inner+Middle, Middle journey", 5000, true, Zone::Inner, Zone::Middle, 10,
{Zone::Middle, 9, 0, Day::Mon, 2026, 10}, "pass,0"},
        {"C02", "Pass Inner+Middle, Outer journey", 20000, true, Zone::Inner, Zone::Middle, 10,
{Zone::Outer, 12, 0, Day::Mon, 2026, 10}, "paid,3000"},
        {"C03", "Pass Inner+Middle, expired month", 20000, true, Zone::Inner, Zone::Middle, 9,
{Zone::Inner, 12, 0, Day::Mon, 2026, 10}, "paid,1000"},
        {"C04", "Pass Middle+Airport, Airport journey", 5000, true, Zone::Middle,
Zone::Airport, 10, {Zone::Airport, 18, 0, Day::Mon, 2026, 10}, "pass,0"},
        {"C05", "No pass, Airport peak, leaves exactly 5000", 12200, false, Zone::Inner,
Zone::Inner, 10, {Zone::Airport, 9, 0, Day::Mon, 2026, 10}, "paid,7200"},
        {"C06", "No pass, Airport peak, leaves 4999", 12199, false, Zone::Inner, Zone::Inner, 10,
{Zone::Airport, 9, 0, Day::Mon, 2026, 10}, "denied"},
        {"C07", "No pass, Airport weekend, leaves exactly 5000", 10400, false, Zone::Inner,
Zone::Inner, 10, {Zone::Airport, 9, 0, Day::Sat, 2026, 10}, "paid,5400"},
    };
    std::cout << "\nPart B: card and pass tests\n";
    for (const CardCase& t : cardCases) {
        SmartCard card(t.balance);
        if (t.hasPass) card.addPass(Pass(t.pa, t.pb, 2026, t.passMonth));
        long long charged = 0;
        Outcome o = card.travel(t.j, Category::Regular, charged);
        report(t.id, t.name, t.expected, outcomeText(o, charged));
    }
```

```
    // ---- worked example: one week of travel on one card ----
    std::cout << "\nPart C: worked example (balance Rs 500.00, pass for Inner + Middle, Oct
2026)\n";
    SmartCard card(50000);
    card.addPass(Pass(Zone::Inner, Zone::Middle, 2026, 10));
    struct Step { const char* label; JourneyInfo j; };
    const Step steps[] = {
        {"Mon Inner 09:00 ", {Zone::Inner, 9, 0, Day::Mon, 2026, 10}},
        {"Mon Outer 18:00 ", {Zone::Outer, 18, 0, Day::Mon, 2026, 10}},
        {"Sat Airport 09:00", {Zone::Airport, 9, 0, Day::Sat, 2026, 10}},
        {"Sun Middle 12:00", {Zone::Middle, 12, 0, Day::Sun, 2026, 10}},
        {"Tue Airport 17:30", {Zone::Airport, 17, 30, Day::Tue, 2026, 10}},
    };
    long long total = 0;
    for (const Step& s : steps) {
        long long charged = 0;
        Outcome o = card.travel(s.j, Category::Regular, charged);
        total += charged;
        std::cout << "  " << std::left << std::setw(18) << s.label << std::right << " charged " <<
std::setw(5)
                  << charged << " paise  (" << outcomeText(o, charged) << "), balance " <<
card.balance() << '\n';
    }
    // hand calculation: 0 + 3600 + 5400 + 0 + 7200 = 16200; 50000 - 16200 = 33800
    report("W01", "Total charged in worked example", "16200", std::to_string(total));
    report("W02", "Final balance in worked example", "33800", std::to_string(card.balance()));

    std::cout << "\nSummary: " << passed << " passed, " << failed << " failed, " << (passed +
failed) << " total\n";
    return failed == 0 ? 0 : 1;
}
```

Compile command used: g++ -std=c++17 -Wall -Wextra -o Task12 Task12.cpp
Compiler messages with -Wall -Wextra: none (the program compiled without warnings).
Actual output of the revised program

**Actual output (from execution)**

```
Part A: fare calculation tests
A01  Airport weekday off-peak (Mon 12:00)        expected 6000     actual 6000     PASS
A02  Airport weekday peak (Mon 09:00)            expected 7200     actual 7200     PASS
A03  Airport weekend off-peak (Sat 12:00)        expected 5400     actual 5400     PASS
A04  Airport weekend peak hour (Sat 09:00)       expected 5400     actual 5400     PASS
A05  Outer weekend peak hour (Sun 18:00)         expected 2700     actual 2700     PASS
A06  Inner weekend off-peak (Sun 12:00)          expected 900      actual 900      PASS
A07  Middle weekday peak 17:00 (old rule)        expected 2400     actual 2400     PASS
A08  Outer weekday 10:00 off-peak (old rule)     expected 3000     actual 3000     PASS
A09  Outer weekday 20:00 off-peak (old rule)     expected 3000     actual 3000     PASS
A10  Student Inner weekend off-peak              expected 450      actual 450      PASS
A11  Senior Airport weekday peak                 expected 4320     actual 4320     PASS

Part B: card and pass tests
```

C01 Pass Inner+Middle, Middle journey expected pass,0 actual pass,0 PASS
C02 Pass Inner+Middle, Outer journey expected paid,3000 actual paid,3000 PASS
C03 Pass Inner+Middle, expired month expected paid,1000 actual paid,1000 PASS
C04 Pass Middle+Airport, Airport journey expected pass,0 actual pass,0 PASS
C05 No pass, Airport peak, leaves exactly 5000 expected paid,7200 actual paid,7200 PASS
C06 No pass, Airport peak, leaves 4999 expected denied actual denied PASS
C07 No pass, Airport weekend, leaves exactly 5000 expected paid,5400 actual paid,5400 PASS

Part C: worked example (balance Rs 500.00, pass for Inner + Middle, Oct 2026)
Mon Inner 09:00 charged 0 paise (pass,0), balance 50000
Mon Outer 18:00 charged 3600 paise (paid,3600), balance 46400
Sat Airport 09:00 charged 5400 paise (paid,5400), balance 41000
Sun Middle 12:00 charged 0 paise (pass,0), balance 41000
Tue Airport 17:30 charged 7200 paise (paid,7200), balance 33800
W01 Total charged in worked example expected 16200 actual 16200 PASS
W02 Final balance in worked example expected 33800 actual 33800 PASS

Summary: 20 passed, 0 failed, 20 total

[Screenshot to be inserted: Task12_Output.png - code and output from the compiler window]
Worked examples (calculated by hand, then checked with the program)

| **Journey** | **Calculation** | **Fare** |
|---|---|---|
| Airport, Monday 09:00 (peak) | 6000 + 6000 × 20 / 100 = 6000 + 1200 | 7200 paise (A02) |
| Airport, Saturday 09:00 (peak hour) | 6000 − 6000 × 10 / 100 = 6000 − 600. No surcharge on a weekend | 5400 paise (A04) |
| Outer, Sunday 18:00 (peak hour) | 3000 − 300 | 2700 paise (A05). Not 3240 (discount on the surcharged fare) and not 3600 |
| Student, Inner, Sunday 12:00 | 1000 − 100 = 900; 900 × 50 / 100 | 450 paise (A10) |
| Senior, Airport, Tuesday 09:00 | 7200 × 60 / 100 | 4320 paise (A11) |
| Pass (Inner + Middle, Oct 2026), Middle journey | covered by the pass | 0 charged (C01) |
| Same pass, Outer journey | not covered, normal fare 3000 | 3000 charged (C02) |
| Same pass, journey in November 2026 | pass month expired, normal fare 1000 | 1000 charged (C03) |
| Card with 12200, Airport peak 7200 | 12200 − 7200 = 5000, exactly the minimum | allowed (C05); with 12199 it is refused (C06) |

**Worked example for one card (Part C of the output).** Balance Rs 500.00 (50000 paise) with a pass for Inner and Middle in October 2026: Monday Inner is covered (0), Monday Outer 18:00 peak is 3600, Saturday Airport 09:00 is 5400 (no surcharge), Sunday Middle is covered (0) and Tuesday Airport 17:30 peak is 7200. Total by hand: 0 + 3600 + 5400 + 0 + 7200 =

16200 paise, and 50000 − 16200 = 33800 paise. The program printed the same total and balance (tests W01 and W02).

D. Evidence-Based Defence (2 Marks)

Test results

| ID | Test case | Expected | Actual | Result |
|---|---|---|---|---|
| A01 | Airport weekday off-peak (Mon 12:00) | 6000 | 6000 | PASS |
| A02 | Airport weekday peak (Mon 09:00) | 7200 | 7200 | PASS |
| A03 | Airport weekend off-peak (Sat 12:00) | 5400 | 5400 | PASS |
| A04 | Airport weekend peak hour (Sat 09:00) | 5400 | 5400 | PASS |
| A05 | Outer weekend peak hour (Sun 18:00) | 2700 | 2700 | PASS |
| A06 | Inner weekend off-peak (Sun 12:00) | 900 | 900 | PASS |
| A07 | Middle weekday peak 17:00 (old rule) | 2400 | 2400 | PASS |
| A08 | Outer weekday 10:00 off-peak (old rule) | 3000 | 3000 | PASS |
| A09 | Outer weekday 20:00 off-peak (old rule) | 3000 | 3000 | PASS |
| A10 | Student Inner weekend off-peak | 450 | 450 | PASS |
| A11 | Senior Airport weekday peak | 4320 | 4320 | PASS |
| C01 | Pass Inner+Middle, Middle journey | pass,0 | pass,0 | PASS |
| C02 | Pass Inner+Middle, Outer journey | paid,3000 | paid,3000 | PASS |
| C03 | Pass Inner+Middle, expired month | paid,1000 | paid,1000 | PASS |
| C04 | Pass Middle+Airport, Airport journey | pass,0 | pass,0 | PASS |
| C05 | No pass, Airport peak, leaves exactly 5000 | paid,7200 | paid,7200 | PASS |
| C06 | No pass, Airport peak, leaves 4999 | denied | denied | PASS |
| C07 | No pass, Airport weekend, leaves exactly 5000 | paid,5400 | paid,5400 | PASS |
| W01 | Total charged in worked example | 16200 | 16200 | PASS |
| W02 | Final balance in worked example | 33800 | 33800 | PASS |

All 20 tests passed (Summary line of the output). The tests include three that repeat the original rules (A07: 17:00 is peak, A08 and A09: 10:00 and 20:00 are off-peak), so the original behaviour was not broken by the change. The boundary of the minimum balance still works with the new fares (C05 and C06 for Airport, C07 for the weekend fare).

Why modify and not retain or redesign

| Option | Assessment |
|---|---|
| Retain | Not possible. The tests above show the original fare function cannot express the weekend rule or the pass, and a forgotten switch case silently gives −1 |
| Modify (chosen) | The parts that were correct and tested (paise, long long, isPeak, the private balance and its rules) are kept, so the earlier 36 tests keep their meaning. Only the fare table, the fare function and the card were extended, and one new class was added. All 20 new tests passed |

| Option | Assessment |
|---|---|
| Redesign | For example, a separate FareRule class hierarchy with virtual functions for weekend, peak and pass rules. This is more flexible if many more rules are added, but for three rules it adds classes and virtual calls with no benefit now and more code to test |

Complexity and trade-offs

| Point | Original design | Revised design |
|---|---|---|
| Time per journey | O(1) | O(1): one table access, a few comparisons and arithmetic operations; Pass::covers() compares two zones and a month, so it is also O(1). A pass with k chosen zones would cost O(k) |
| Space per card | O(1): one balance | O(1): the balance and an optional pass (two zones, a month and a year) |
| Fare table | 3 constants | Array with Z = 4 entries (32 bytes), O(Z) |
| Adding a fifth zone | New case in the switch and a new constant | New enumerator and one new value in the table. The size follows from Zone::Count |
| Forgotten update | Silent −1 fare | A zone beyond the table would be out of range; the table size is tied to Zone::Count, and a static_assert could compare the table length with it |
| Testability | One table of tests | Two tables (fares, cards and passes), all with expected values worked out by hand |

**Limitations of the revised design.** The weekend, the concession order and the pass rules follow my assumptions listed at the start of this task. The date is passed in as a day-of-week, a month and a year, so there is no calendar calculation, no holiday list and no pass purchase or expiry processing. A pass is tied to a calendar month only. These could be added without changing the fare table.

12.2 Conclusion

The original design was modified, not thrown away. The fare table indexed by zone, the separate discount rule, the Pass class and the optional pass in SmartCard meet the three new requirements, keep the old rules (verified by three regression tests) and cost the same O(1) time per journey.

Summary of Results and Final Conclusion

| Task | Programs | Result |
|---|---|---|
| 1 | Task01_ClassDesign, Task01_Procedural_sketch | Compiled and ran. Polymorphic fares for the same journey: Rs 36.00, Rs 18.00, Rs 21.60 |
| 2 | Task02_Constants, Task02_COMPILEFAIL | Macro pitfall (5 instead of 8) shown; enum class conversions rejected by the compiler |
| 3 | Task03_DataTypes | sizeof and climits printed; int overflows for 3,000,000 journeys; double rounding error measured |

| Task | Programs | Result |
|---|---|---|
| 4 | Task04, Task04_fixed, Task04_narrowing, Task04_COMPILEFAIL | Prediction matched; sign-compare warning shown; brace narrowing rejected |
| 5 | Task05, Task05_incdemo, Task05_peaktable | Prediction matched; i = i++ warning shown; peak table agrees for all 24 hours |
| 6 | Task06, Task06_boundary, Task06_array | Output identical to the expected output; 10:00 and 20:00 off-peak confirmed |
| 7 | Task07_Procedural, Task07_OOP, Task07_PublicMember, Task07_COMPILEFAIL | OOP version refuses invalid changes; procedural and public-member versions allowed a negative balance |
| 8 | Task08_cin, Task08_getline | Recovery from letters and out-of-range input shown; 7x handled differently by the two methods |
| 9 | Task09 | 36 of 36 tests passed |
| 10 | Task10 | Exact total 4,180,000,000 paise; double differs by 0.039 paise; int overflows for the network |
| 11 | Task11 (with fares.cfg and fares_bad.cfg) | Configuration loaded and validated; the invalid file was refused |
| 12 | Task12, Task12_impact, Task12_COMPILEONLY_nodefault | 20 of 20 tests passed for the revised design |

Final conclusion. The fare problem was analysed as five objects with clear rules, and the data types, constants, operators and control structures were chosen on the evidence of the experiments: integer paise in long long removes the one-paisa errors and the overflow of the network total, constexpr constants and an enum class keep the fares in one place with type checking, an exact half-open interval fixes the 10 AM and 8 PM confusion, and a SmartCard class with a private balance makes a negative balance impossible. The fare logic passed all 36 tests of Task 9. When the fourth zone, the weekend discount and the monthly pass were added in Task 12, the design was modified (not rewritten) and passed all 20 new tests together with the old rules.
Limitations: the concession percentages and the weekend and pass details are assumptions; screenshots and the GitHub repository have to be added by the student; and the timing numbers come from one machine without compiler optimisation.